



Infor Education
Master Infor products.
Maximize your potential.

CloudSuite Industrial: v10 Creating Form Personalizations Training Workbook (10.18)

CloudSuite Industrial

October 10, 2018

Course code: 01_0011018_IEN3552_SYT

Legal Notice

Copyright © 2018 Infor. All rights reserved.

Important Notices

The material contained in this publication (including any supplementary information) constitutes and contains confidential and proprietary information of Infor.

By gaining access to the attached, you acknowledge and agree that the material (including any modification, translation or adaptation of the material) and all copyright, trade secrets and all other right, title and interest therein, are the sole property of Infor and that you shall not gain right, title or interest in the material (including any modification, translation or adaptation of the material) by virtue of your review thereof other than the non-exclusive right to use the material solely in connection with and the furtherance of your license and use of software made available to your company from Infor pursuant to a separate agreement, the terms of which separate agreement shall govern your use of this material and all supplemental related materials ("Purpose").

In addition, by accessing the enclosed material, you acknowledge and agree that you are required to maintain such material in strict confidence and that your use of such material is limited to the Purpose described above. Although Infor has taken due care to ensure that the material included in this publication is accurate and complete, Infor cannot warrant that the information contained in this publication is complete, does not contain typographical or other errors, or will meet your specific requirements. As such, Infor does not assume and hereby disclaims all liability, consequential or otherwise, for any loss or damage to any person or entity which is caused by or relates to errors or omissions in this publication (including any supplementary information), whether such errors or omissions result from negligence, accident or any other cause.

Without limitation, U.S. export control laws and other applicable export and import laws govern your use of this material and you will neither export or re-export, directly or indirectly, this material nor any related materials or supplemental information in violation of such laws, or use such materials for any purpose prohibited by such laws.

Trademark Acknowledgements

The word and design marks set forth herein are trademarks and/or registered trademarks of Infor and/or related affiliates and subsidiaries. All rights reserved. All other company, product, trade, or service names referenced may be registered trademarks or trademarks of their respective owners.

Table of contents

About this workbook.....	7
Course overview.....	9
Course description and agenda.....	10
User interface	14
Navigation pane menus	14
Main toolbar.....	14
Keyboard shortcuts.....	14
Field types	18
Icon name differences.....	18
Lesson 1: Exploring the data architecture	21
Review how forms, fields, records, and collections work in the system	22
Forms.....	22
Fields.....	22
Records.....	22
Collections.....	22
Understand the relationship between form metadata, IDOs, and the application database.....	23
How Intelligent Data Objects (IDOs) work with forms.....	23
How forms use IDOs.....	23
The three-tier architecture in the system	25
Check your understanding	26
Lesson 2: Setting up form editing permissions	27
Explore the scope of form editing	28
Vendor version: ScopeType 0	28
Site Default version: ScopeType 1	28
Group version: ScopeType 2.....	28
User version: ScopeType 3.....	28
Understand edit permissions.....	30
Review the Vendor Developer edit permission	33
Review the Site Developer edit permission.....	34
Review the Full User edit permission	35
Review the Basic User and None edit permissions.....	36
Basic user.....	36
None	36
Exercise 2.1: Log in to CloudSuite and assign form editing permission	36
Check your understanding	39
Lesson 3: Editing forms	40
Change default font	41
Changing default fonts using the User Preferences dialog box	41
Using the form and component sheets to change default fonts.....	43
Regenerating a form	43
Set form and component background colors	44
Using the form and component properties to set background color.....	44
Using the Edit Color dialog box to set background color.....	45
Set a background image	47
Importing background images into the system.....	47
Setting a background image on a form.....	47
Setting a banner	48
Change some basic component properties.....	49
Setting a field to read-only.....	49

Hiding a component or field.....	49
Setting a field to be dynamically disabled or enabled	50
Using images in components	50
Formatting date, time, and other specific data in components.....	51
Formatting input masks in components	51
Revert to the default version of a form.....	52
Copying a form	52
Reverting a form	52
Demo: Change font on form.....	53
Exercise 3.1: Change font on form	53
Demo: Add background color to form	54
Exercise 3.2: Add background color to form	54
Demo: Set a background image.....	55
Exercise 3.3: Set a background image	55
Check your understanding	56
Lesson 4: Customizing form components	57
Explore the system components	58
Add components to a form	60
Set up components on a form	61
Sizing a component	61
Aligning the components.....	61
Set tab order for components.....	63
Using Tab Order to set tabs for components	63
Omitting a component from tab order	64
Identifying tab order tags.....	64
Demo: Add components to a form.....	64
Scenario	64
Exercise 4.1: Add components to a form	65
Check your understanding	66
Lesson 5: Using inherited attributes	67
Understand the value of inherited attributes	68
Use component classes.....	71
Creating component classes	71
Applying inherited attributes to a component	71
Use property class extensions	73
Bind components to an IDO property	74
Binding a component	74
Binding components to properties	74
Setting data bindings for a component	75
Demo: Associate inherited attributes.....	75
Exercise 5.1: Associate inherited attributes	75
Check your understanding	77
Lesson 6: Setting up list sources	78
Create a list source	79
Create an IDO list source.....	80
Create an inline list	82
Demo: Add combo boxes and input list source information.....	83
Exercise 6.1: Add combo boxes and input list source information	83
Check your understanding	86
Lesson 7: Adding validators to components	87
Explore the use of validators in the system.....	88
Setting validator properties	88

Using validator messages	88
Understanding the most common types of validators.....	88
Considerations when using validators	89
Create, copy, edit, or delete validators	90
Assign a validator to a component	90
Demo: Add an inline list validator to the combo box form component.....	91
Exercise 7.1: Add an inline list validator to the combo box form component.....	91
Check your understanding	92
Lesson 8: Creating and editing strings	93
Explore the use of strings in the system	94
Examples of strings	94
Use the static component.....	95
Create a translatable string	96
Copy, edit, or delete strings.....	97
Demo: Create static form components and strings	97
Exercise 8.1: Create static form components and strings.....	97
Check your understanding	99
Lesson 9: Adding user-extended tables (UETs) fields and user-defined type lists.....	100
Add user-extended table fields.....	101
Adding user-extended table fields	101
Create and use a new user-defined type list.....	102
Demo: Add UET field and data.....	102
Exercise 9.1: Add UET field and data.....	103
Demo: Work with a UET.....	105
Exercise 9.2: Work with a UET	105
Check your understanding	107
Lesson 10: Setting up event handlers.....	108
Explore the use of events and event handlers in the system.....	109
Understanding event handlers	110
Event handler sequence	110
Built-in responses	110
Event handler messages.....	111
Create, copy, edit, or delete event handlers	112
Create, copy, or delete event handlers for a form	112
Edit an event handler.....	112
Demo: Create a new button form component and add an event handler.....	113
Exercise 10.1: Create a new button form component and add an event handler	114
Check your understanding	116
Lesson 11: Form Sync	117
Why you need FormSync.....	118
Overview	118
The Nature of Customizations.....	118
The relationship between Global Objects and Forms during customization.....	118
The Basic Synchronization Process	119
Conflicts Between Customizations and New Versions	119
Automatic Conversions of Scripts.....	119
Scope of Synchronization.....	120
Planning Customization and Synchronization	120
FormSync Administration Utilities.....	120
Synchronizing Site Default or Group Versions.....	121
For More Information	121
Exercise 11.1: Form Sync the Price Matrix form	121

Check your understanding	124
Course summary	125
Course review.....	126
Appendices.....	127
Appendix A: User accounts.....	128
Appendix B: Creating a new form.....	129
Copy a form.....	129
Create a new form	129

About this workbook

Welcome to this Infor Education course! We hope you will find this learning experience enjoyable and instructive. This Training Workbook is designed to support the following forms of learning:

- Classroom instructor-led training
- Virtual instructor-led training
- Self-directed learning

This Training Workbook is not intended for use as a product user guide.

Activity data

You will be asked to complete some practice exercises during this course. Step-by-step instructions are provided in this guide to assist you with completing the exercises. Where necessary, data columns are included for your reference.

Your instructor will provide more information on systems used in class, including server addresses, login IDs, and passwords.

Self-directed learning

If you are taking this course as self-directed learning, there may be instructor-recorded presentations and/or simulations available to assist you.

If instructor-recorded presentations are available, a hyperlink to the recording will be included on the first page of each corresponding Lesson.

If simulations are available, the demos and exercises throughout this Training Workbook will include hyperlinks that allow you to view and/or practice the execution of the demo or exercise in a simulated training environment.

Learning Libraries

Learning Libraries in Infor Campus include learning materials that are available to you online, anytime, anywhere. These materials can supplement instructor-led training, providing you with additional learning resources to support your day-to-day business tasks and activities.

Please note that if you accessed this Training Workbook directly via a Learning Library, you will not have access to the Infor Education Training Environment that is provided with all instructor-led and most self-directed learning course versions, as referenced above. Therefore, you will not be able to practice the exercises in the specific Training Environment for which the exercises in this Training Workbook were written.

Symbols used in this workbook



Exercise



Your notes



Question



Demo



Important note



Answer



Scenario or Discussion



Critical note



Task simulation



For your reference



Navigation back



Navigation down



Course overview

Estimated time

.25 hours

Learning objectives

Upon completion of this course, you will be able to:

- Identify the elements of the system data architecture.
- Identify how to assign different levels of form editing permission to specific users.
- Describe how to apply editing attributes to a form.
- Explain how to use the inherited attributes.
- Identify the how to create list sources, IDO list sources, and inline lists.
- Describe the purpose of validators and how they are used in forms.
- Explain how to create and edit strings.
- Describe how to add user-extended tables (UETs) fields and user-defined type lists.
- Explain how to apply an event handler to a form.
- Explain how to use FormSync.

Topics

- Course description and agenda

Course description and agenda

This course covers the basic formatting options used to personalize forms in the system and includes instructions on how to assign formatting permissions, edit forms, as well as how to create components, strings, and event handlers.

Course duration

8 hours

Prerequisite knowledge

To optimize your learning experience, Infor recommends that you have the following knowledge prior to attending this course:

- Knowledge of how data is structured and organized
- Ability to navigate the user interface

Audience

- Customer User
- Pre-Sales Consultant
- Business Consultant
- Technical Consultant
- Support
- System Administrator

System requirements

- CloudSuite Business / CloudSuite Industrial® Training Environment

Reference materials

Reference materials are available from the following locations:

- Help menu
- Infor Xtreme®

Course agenda

The agenda below details the contents of this course, including lesson-level learning objectives and supporting objectives.

Lesson	Lesson title	Learning objectives	Estimated time (hours)
Course overview		Review course expectations.	.25
1	Exploring the system data architecture	• Explain how the form metadata, Intelligent Data Objects (IDOs), and the application database relate to one another. • Describe the inheritance hierarchy and the three-tiers of the application structure of the Infor Mongoose framework.	.5
2	Setting up form editing permissions	• Describe how form editing scope impacts permission levels in the system. • Explain the different form editing permission levels. • Identify the steps to assign different levels of form editing permissions to users.	.5
3	Editing forms	• Describe how to change fonts. • Describe how to create and edit background color. • Explain how to set a background image. • Describe how change basic default properties. • Explain how to revert to the default version of a form.	1
4	Customizing form components	• Identify different types of components in system forms. • Explain how to add components to forms. • Explain how to manipulate the alignment, size, and location of a component on a form.	1

Lesson	Lesson title	Learning objectives	Estimated time (hours)
		Identify the steps to set the tab order of components.	
5	Using inherited attributes	- Explain the value of inherited attributes. - Describe how to use the inherited attributes. - Explain how property class extensions are applied to a component. - Describe how to bind components to an IDO property.	.5
6	Setting up list sources	- Explain the purpose of a list source. - Explain the purpose of IDO Collection. - Describe how to use inline lists.	1
7	Adding validators to components	- Explain the purpose of validators. - Identify the types of validators. - Describe how to create a validator.	.5
8	Creating and editing strings	- Explain how strings are used. - Describe how the static component is used in strings. - Describe how to create a string. - Describe how to view and edit a string.	.75
9	Adding user-extended tables fields (UTES) and user-defined type lists	- Identify the purpose of user-extended tables (UTs). - Explain how to enter and use the user-defined type list feature.	.75
10	Setting up event handlers	- Explain the purpose of event handlers. - Identify the types of event handlers that are used in forms. - Describe how to add an event handler to a form.	.75
11	Form Sync	- Explain the purpose for Form Sync. - Identify the Key areas of Form Sync.	.75

12 Course overview

Lesson	Lesson title	Learning objectives	Estimated time (hours)
		Describe how to carry out Form Sync.	
Course summary		Review course objectives.	

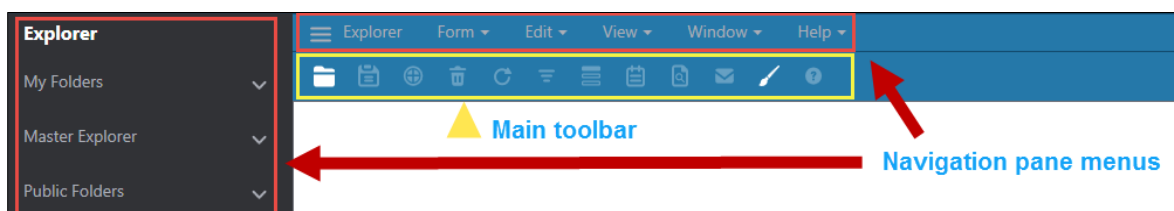
Appendices

This section contains information that is not part of the instructional content of this course, but provides additional related reference information.

Appendix	Appendix title	Content description
Appendix A	User accounts	This appendix provides a reference for student login credentials.
Appendix B	Creating a new form	Copying and creating forms

User interface

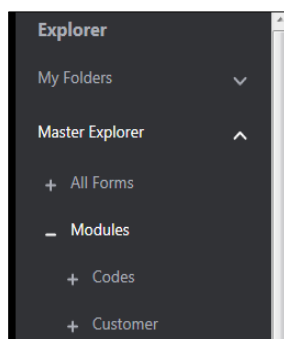
Standard user interface elements include the navigation pane menus and the main toolbar.



Navigation pane menus and main toolbar

Navigation pane menus

You can browse all available folders through the navigation pane menus and submenus. Click the down arrow for each of the navigation pane menu, this allows you to explore the submenus contained beneath.



Navigation pane menus and submenus

For information on how to pin submenus to the navigation pane, please refer to the Creating Form Personalizations workbook or ILT. For example, if you know that you will be using the Customer Service submenus frequently, you can display the menus at the top level, by pinning them to the navigation pane.

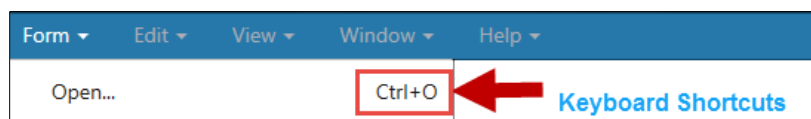
Main toolbar

The main toolbar contains icons that allow you to quickly complete commonly used tasks, such as saving data or closing a form. You can position your mouse pointer over an icon on the toolbar to reveal its description and functionality.

To view a list of all toolbar icons and for steps on how to add/remove icons from the default toolbar, See Appendix B. **Note:** Availability to make hiding buttons visible is not available for Web users.

Keyboard shortcuts

Some navigation pane submenus display available keyboard shortcuts that can be used to save multiple steps when completing certain tasks.



The following tables display the available shortcut keys and their actions.

Navigating forms

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
Close current form	Form > Close and Cancel Changes	Ctrl + F4	N/A		Hidden
Open Select Form	Form > Open	Ctrl + O	Ctrl + O		Visible
Open workspaces	Form > Workspaces	Ctrl + W	N/A		
Switch view to next open form		Ctrl + Tab	N/A		

Navigating records

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
Move to first editable field of form	View > Home Cursor	Ctrl + Home	Ctrl + Home		
Move to the next field		Tab	Tab		
Move to the previous field		Shift + Tab	Shift + Tab		
Move to the next record	Actions > Next	F8	F8		Hidden
Move to the previous record	Actions > Previous	F7	F7		Hidden
Switch the next collection on the form to become the current collection in a form that contains multiple collections.	View > Activate Next Collection	Ctrl + F8	N/A		Visible
Hide/Show 1 st Splitter Pane. Toggles back and forth between form and dual view.	View > Hide/Show 1 st Splitter Pane	Ctrl + 1	Ctrl + Shift + 1		
Hide/Show 2 nd Splitter Pane. Toggles back and forth between grid and dual view.	View > Hide/Show 2 nd Splitter Pane	Ctrl + 2	Ctrl + Shift + 2		

Editing records

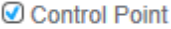
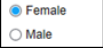
Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
Activate the drop-down list with an implied asterisk (i.e. wildcard) after the entered text in the field to then populate the list with data entries that begin with the entered text.		F2	F2		
Add a new record Note: The Actions menu is only available when a form is open.	Actions > New	Ctrl + N	N/A		Visible
Add value for current field	Edit > Add	Ctrl + A	N/A		Hidden
Copy (only works to copy a record with web client)	Actions > Copy	Ctrl + C	Ctrl + C		
Cut		Ctrl + X	Ctrl + X		
Paste		Ctrl + V	N/A		
Undo	Edit > Undo	Ctrl + Z	N/A		
Delete record	Actions > Delete	Ctrl + D	Ctrl + D		Visible
Find value for current field (only available for certain drop-down fields)	Edit > Find Value for Current Field	Ctrl + F	N/A		Hidden
Display details for current field drill-down (only available for certain drop-down fields)	Edit > Details for Current Field	Ctrl + L	N/A		Hidden
Save changes (only available when a form is open)	Actions > Save	Ctrl + S	Ctrl + S		Visible

Filtering records

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
In Filter Place mode, cancel the filter in place and return to the collection previously displayed	Actions > Filter > Cancel in Place	F3	F3		
In Refresh/Run mode, begin filter in place (i.e. clear the collection and go back to Filter In Place mode) In Filter In Place Mode, execute in place (i.e. run the filter)	Actions > Filter > Begin in Place Actions > Filter > Execute in Place	F4	F4		Visible
In Refresh/Run mode, refresh the current collection. In Filter In Place Mode, clear the filter in place.	Actions > Refresh Actions > Filter > Clear in Place	F5	F5		Visible
In Refresh/Run mode, refresh the current record.	Actions > Refresh current	Ctrl + F5	Ctrl + F5		Visible
In Refresh/Run mode, repeat Find Value in Collection	Edit > Repeat Find	Ctrl + F2	Ctrl + F2		
In Refresh/Run or New mode, open associated query form to specify filter criteria.	Actions > Filter > By Query	Ctrl + Q	N/A		
Open field level help topic	Help > Current Field	F1	N/A		
Toggle design mode on and off.	Edit > Design Mode	Ctrl + E	N/A		Visible
Print	Print	Ctrl + P	N/A		

Field types

The following tables display the field types available with a brief description:

Field type	Description
Required, system-generated, and read-only fields	
Order Date: 	- Any field that is required will have a red star - A system-generated field will have a green star - A field with no border around the data is read-only
Standard GUI fields	
	Check box indicating Yes or No
	Radio button to select one option from multiple choices
Order Type: 	Drop-down list to select one option from multiple choices

Icon name differences

Here are the names of the different icons in the smart client and web client.

Smart Client	Web Client
Open a form	Open
Save	Save
Create a new object in the current collection	New
Delete an object in the current collection	Delete
Refresh the current collection	Refresh
Filter in Place	Filter in Place
Get more rows in the current collection	Get more rows
Display notes for the current object	Notes
Display documents for the current object	Documents
Send email for the current object	Send email for the current object
Export the data in the current collection to excel	Export to excel
Enter or exit design mode on this form	Open designer in a new window
Translate	Translate

Smart Client	Web Client
Get help on the current form	Help Current Form



Lesson 1: Exploring the data architecture

Estimated time

.5 hours

Learning objectives

In this lesson, you will:

- Explain how the form metadata, Intelligent Data Objects (IDOs), and the application database relate to one another.
- Describe the inheritance hierarchy and the three-tiers of the application structure of the Infor Mongoose framework.

Topics

- Review how forms, fields, records, and collections work in the system
- Understand the relationship between form metadata, IDOs, and the application database
- Compare the three-tier inheritance architecture in the system to the car manufacturing process

Review how forms, fields, records, and collections work in the system

Mongoose is Infor's development framework which enables upgradeable extensibility, cloud on premise and hybrid deployments, and rapid application development, among other things. The system is built on the Infor Mongoose framework.

System data is stored in tables in its main application database. To access or submit information, you generally should use one or more tables in the databases. The system uses fields and other components on forms to present collections of data records.

Forms

A form is the window through which you interact with information in the system. Each form is devoted to a business task or to several related tasks.

For example, you can use an **Attendance** form to record and track employee work attendance records. Then you can view and manipulate the attendance information by using certain features and functionalities in the system.

Fields

A field is an area on a form that shows you information that is related to the other fields in a form. It is considered a component of a form as are radio boxes, buttons, and other widgets. The information in a field is being pulled from a database where it resides. You can also enter and save new data in fields. Mongoose components can also be other types of controls like notebook tabs and buttons.

Forms usually have several fields that all display related pieces of information. For example, each field in the **Attendance** form displays different but related information such as:

- Employee (ID number)
- Employee name (not labeled)
- Date
- Hours

Records

In Mongoose, records are rows on a collection and collections are provided by an IDO. So each piece of data on a record is provided by a property on the IDO which is the source of the collection.

An example of a record would be viewing information on an employee that is out on leave. The appropriate fields together that contains information about the employee's absence would be a record.

Collections

In Mongoose, collections are sets of records provided by an IDO, where each record has values from an IDO property. A form can define 0 or more collections. Grids and other types of components can be bound to collections on forms, and other types of components can be bound to properties on collections on forms, like edits or combos.

Understand the relationship between form metadata, IDOs, and the application database

Now that you have reviewed how forms, components, and collections work in the system, let's look at the data architecture. It is important that you have a general understanding of how the data on the forms relate to specific IDOs, and how specific IDOs relate to the data in the application database. We will start with a detailed definition of an IDO.

How Intelligent Data Objects (IDOs) work with forms

Intelligent Data Objects (IDOs) work with form metadata and the application database in the system. The application database is linked to an objects database and a forms database through configurations. An IDO is a business object that contains units of information and logic that are called from the client layer to interact with the data in the database. The job of the IDO is to transport collections of data back and forth, with any validation or rules needed, between the client and the database.

An IDO consists of these elements:

- A reference to one or more SQL tables
- A set of property definitions
- A set of zero or more methods
- An optional reference to an IDO Extension Class

In a set of one or more SQL tables each table contains the data for a specified part of the application and must include columns (properties) that the system requires to work properly. IDO Properties can be bound to columns on tables referenced by the IDO, calculated values, values providing data to server-side code during query or update, or relationships to child IDOs.

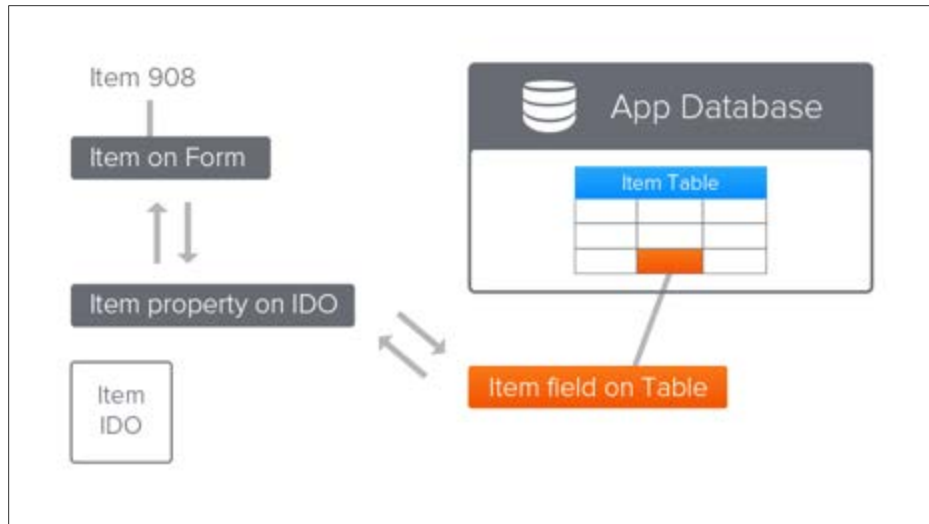
The Mongoose framework provides standard functionality for all IDOs:

- LoadCollection retrieves a collection of rows from the database.
- UpdateCollection takes a set of rows marked for insert, update, or delete, and executes the appropriate SQL code against the database.
- GetPropertyInfo returns detailed information about the properties supported by the IDO.
- Invoke allows you to execute a custom method.

How forms use IDOs

Forms utilize IDOs in many ways. Forms that interact with the application database data define collections based on IDOs. There are many types of validators and list sources that are built over IDOs.

The comparisons on the next pages will help you to better understand the relationship between form data, IDOs, and the application database in the system.

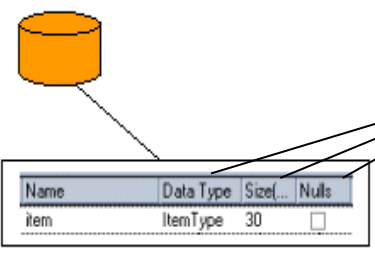
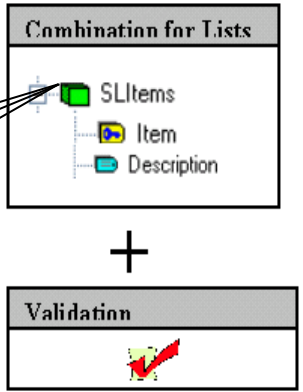
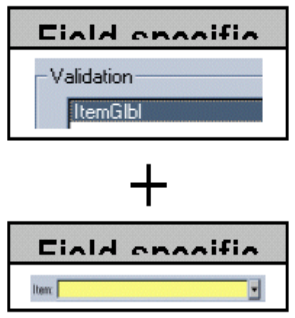


Form data, IDO, and application database relationship

When the **Item** IDO is created:

- It takes on, or inherits, the **Item** field and its attributes such as length, nullability, and data type, along with all the other fields on the table and their attributes.
- Each field in the table may be bound by a property of the IDO.
- The properties are available to a form component to use, in the example above, as the list-source for a drop-down box.

The three-tier architecture in the system

Tier 1: The database	Tier 2: The middle tier IDO	Tier 3: The form user
When a column is added to a table in the database, it is given certain attributes and properties. 	The middle tier IDO defines properties bound to columns on tables and properties that may be calculations. 	Components on the form can be bound to IDO properties, inheriting attributes from the property. The user/developer can override or add more attributes to a component on the form. 

Now that you have a better understanding of the three tiers, it should help you to make informed decisions about when and how you modify or add components to your forms.



Remember: There are many reusable components in the system. Before you add a component, or make a change to a component, make sure you utilize pre-existing work. This will save time and reduce the size of the application considerably.

We will look at how you may reuse Mongoose functionality such as property class extensions and component classes in future lessons.

Check your understanding



An **IDO** consists of these elements, select all that apply:

- a) A reference to one or more SQL tables
- b) A set of property definitions
- c) A set of zero or more methods
- d) An optional reference to an IDO Extension Class
- e) None of the above



The Mongoose framework provides standard functionality for all IDOs such as **LoadCollection**, **UpdateCollection** and **GetPropertyInfo** to name a few:

- a) True
- b) False



When an IDO is created, select all that apply:

- a) It takes on, or inherits, attributes such as length, nullability, and data type, along with all the other fields on the table and their attributes.
- b) Each field in the table may be bound by a property of the IDO.
- c) The properties are available to a form component to use.
- d) None of the above



Lesson 2: Setting up form editing permissions

Estimated time

.5 hours

Learning objectives

In this lesson, you will:

- Describe how form editing scope impacts permission levels in the system.
- Explain the different form editing permission levels.
- Identify the steps to assign different levels of form editing permissions to users.

Topics

- Explore the scope of form editing
- Understand edit permissions
- Review the Vendor Developer edit permission
- Review the Site Developer edit permission
- Review the Full User edit permission
- Review the Basic User and None edit permissions

Explore the scope of form editing

The system manages form editing through permissions. Each user account has an edit permissions setting which determines what kinds of form changes, if any, the user can make. Form changes are made at various levels, called scopes. It is important to understand at what levels a form can be modified. Modifications have a very definite order of precedence.

Let's look at how permissions are structured in the system by examining the higher-level concept of precedence of scope types.

Vendor version: ScopeType 0

The vendor version of forms and other global objects is the version that is shipped to the customer. This version can only be created and modified by a user with the Vendor Developer edit permissions. In the forms database, this shows as **ScopeType 0**.

Site Default version: ScopeType 1

The **Site Default** version of forms and global objects is an override to any vendor version, and will be seen by any user accessing the form, unless there is an applicable group or user version.

Only users with Site Developer edit permissions can create or change **Site Default** versions. **Site Default** versions show as **ScopeType 1** in the forms database.

Group version: ScopeType 2

The **Group** version of forms and global objects is created for a specific group. Users who have that group as their primary permissions group will see that version instead of any site or vendor version.

Only Site Developers can create or modify **Group** versions. **Group** versions show as **ScopeType 2** in the database, with the column **ScopeName** specifying the group to which the change applies.

User version: ScopeType 3

The **User** version of forms and global objects is applied to a specific user. Users who have a **User** version of a form will see that instead of any group, site or vendor version.

Users with edit permissions of Site Developer, Full User, or Basic User can create and edit user versions, but users with Full User or Basic User edit permissions can only create and edit user versions for themselves. User versions show as **ScopeType 3** in the database, with the **ScopeName** column containing the username to which the change applies.

Below is an example of how a user may experience form personalization in the system.



Example of how personalization works in the system

For example: If Bob doesn't have a personalized form, he sees his group's default. If there is no group default, then he sees the site default. If no site default has been set, then he sees the vendor default.

Understand edit permissions

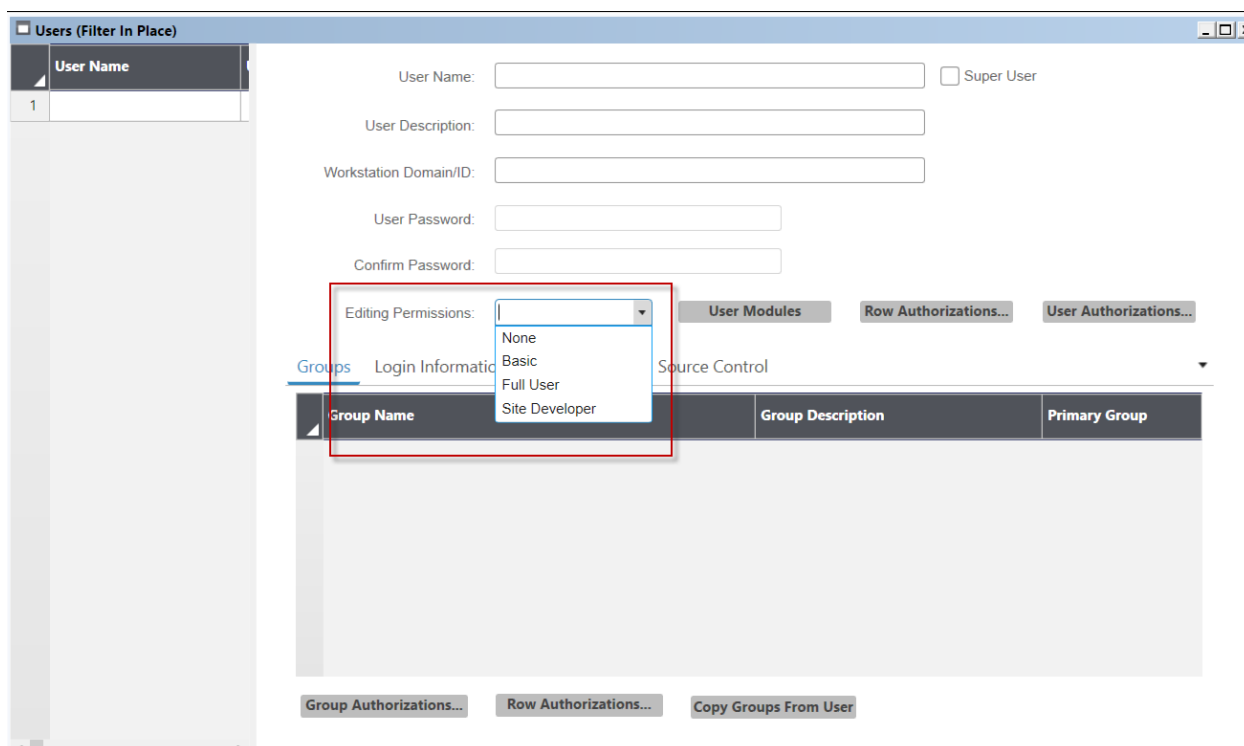
The ability to create or modify a version of a form depends on the edit permissions you are assigned as a user and may be restricted as to which forms you can design. Now that you know the different levels and scope that customizations can fall under, it is important to understand the available edit permissions.

Let's examine the edit permissions that are available in the system:

- Vendor Developer
- Site Developer
- Full User
- Basic User
- None

The edit permission is assigned on the **Users** form. This would typically be done by the system administrator who can also be a Site Developer. Only the Site Developer can specifically change their editing scope by accessing **Scope** which is located on the **Identify Your Editing Scope** dialog box.

As a Site Developer, this is where you will define your editing abilities. You will see the available options and can choose to keep the **Site Default** permission or select the **Group** or **User** permission.

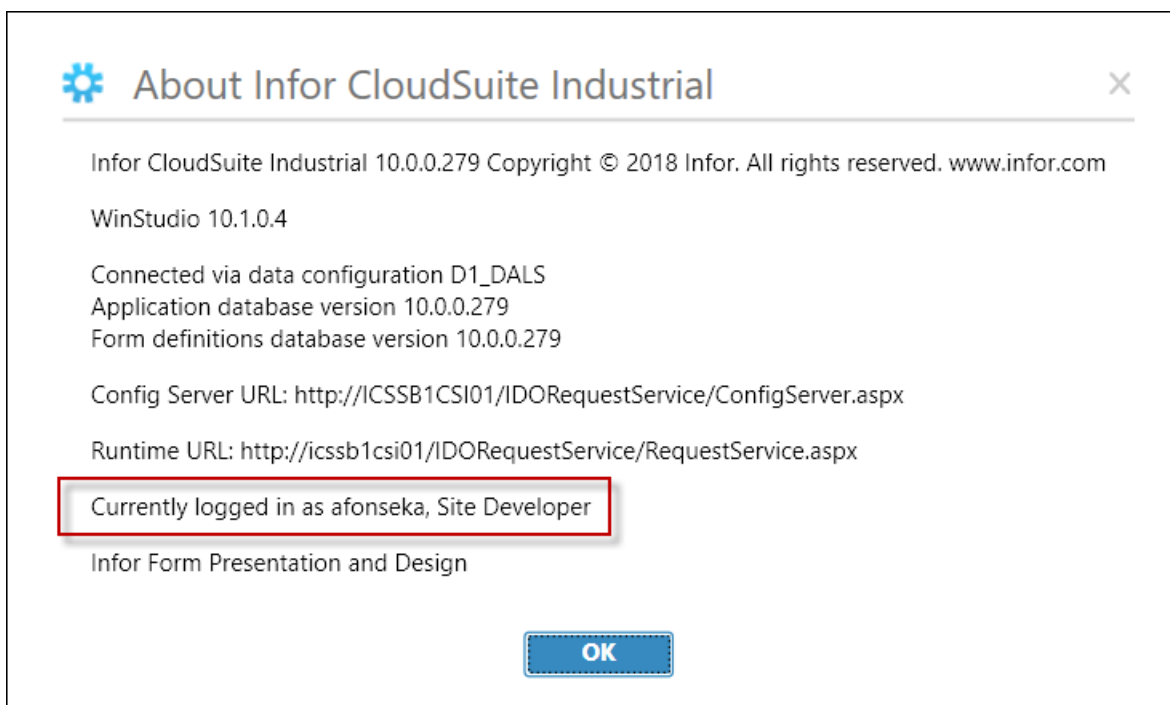
The screenshot shows the 'Users (Filter In Place)' form. On the left is a table with 'User Name' and a single row with the number '1'. The main form area contains fields for 'User Name', 'User Description', 'Workstation Domain/ID', 'User Password', and 'Confirm Password'. There is a 'Super User' checkbox. Below these fields is the 'Editing Permissions' dropdown menu, which is open and shows four options: 'None', 'Basic', 'Full User', and 'Site Developer'. The 'Site Developer' option is highlighted. To the right of the dropdown are buttons for 'User Modules', 'Row Authorizations...', and 'User Authorizations...'. Below these buttons is a 'Source Control' dropdown menu. At the bottom of the form are buttons for 'Group Authorizations...', 'Row Authorizations...', and 'Copy Groups From User'. A table with columns 'Group Name', 'Group Description', and 'Primary Group' is visible at the bottom, but its content is mostly obscured by the dropdown menu.

Form editing permissions



All changes are saved at that permission level until you log off or until there is a change to your editing scope.

In the system, you can check your edit level by logging in then selecting **Help > About <servername>** from the main menu options.



Current permission level screen

Edit permissions also control your access to the Explorer data, the menus that fall under the top-level headings of Master Explorer, Public Folders, User Folders, and My Folders.

You will find the following folders when you access the Explorer window:

- My Folders
- Master Explorer
- Public Folders

Each folder serves a purpose. Let's review their purpose below:

- The **My Folders** folder is created automatically when you log on. You can display and modify your own **My Folders** folder.
 - For example, you can create new folders, copy or delete forms, and change the title of a form that is in your **My Folders** folder. Because you can modify the contents that are in your **My Folders** folder, any changes that the Site Developer makes to the folder or its contents are not secure since they can be modified by you.
 - At the Site Developer level, there are user folders for each user. These folders are created upon user login. Your user folder will mirror your **My Folders** folder and is displayed only to the Site Developers. Site Developers can edit user folders, and any changes are reflected in the user's **My Folders** folder.
 - For example, a form can be copied into your user folder. You will see the copied form in your **My Folders** folder.
- The **Master Explorer** folder contains the folders created by the Vendor Developer and the **All Forms** folder which is a read-only folder that lists every form in the forms database.
- The **Public Folders** folder is initially empty but the Site Developers can create public folders and make them available to all users.

- For example, a form can be distributed to all users or to a group of users if the Site Developer copies it into a public folder.

Review the Vendor Developer edit permission

Developers creating an application have Vendor Developer edit permissions, which are set as a value 4 in the **Users** form. Forms and global objects edited by a Vendor Developer are saved as the vendor default versions, and are intended as the version shipped to the customer.

In the system, you will not see the Vendor Developer option on the Users form's edit permissions drop-down list. However, you can create a Vendor Developer user for problem analysis purposes by entering 4 for the edit permissions of a user on the Users form. When you log in as such a user, you should never make changes to forms because those changes will be overwritten during upgrade.

However, logging in as such a user can be helpful because all customizations will be ignored during the session and only the original vendor version of forms will be used. Therefore, if you are trying to diagnose whether a problem is due to a customization, you can log in as a Vendor Developer and see if the problem occurs.

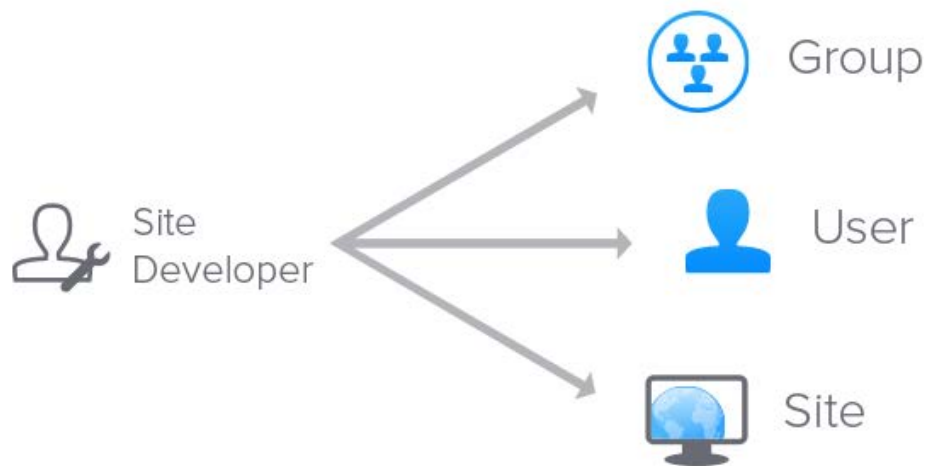
Review the Site Developer edit permission

The user with Site Developer edit permission can edit and create changes at the group, user, or site default level. The Site Developer level is typically assigned to power users such as support or other information technology users.

As a Site Developer, you can perform the following tasks in the system:

- Create or delete forms at the site default, group, and user scope.
- Customize vendor forms in any fashion, making changes at the site, group, or user scope.
- Examples of customizations can be:
 - Adding or removing buttons, tabs, text boxes, validations, event handlers, and grid controls.
 - Using all basic customization options such as hiding or making a field read-only, changing colors and fonts as well as rearranging fields on a form.
- Customize your own **My Folders** folder
- Customize user folders
- View **Master Explorer**
- Customize **Public Folders**

The Site Developer can impersonate any user or group member in the system and then view or edit the specific user or group forms. Below is an example of how a Site Developer may use the system.



Example of Site Developer permission in the system

For example: As the Site Developer, you want to make changes to the **Purchase Order Line Items** form. You can choose to make the changes apply to the Purchasing group, a particular user, or the entire site.

Review the Full User edit permission

The Full User edit permission allows you to make any changes to forms or global objects though your changes are always user versions for yourself only. You cannot create new forms or delete existing ones, though you can revert to the next applicable version of forms.

As a Full user, you:

- Can customize your own forms by adding or removing:
 - Buttons, tabs, and text boxes.
 - Validation and event handlers.
 - Grid controls.
- Can perform all basic customization options such as:
 - Hiding or making a field read-only.
 - Changing colors and fonts on a form and rearranging fields on a form.
- Can customize your own **My Folders** folder.
- Can view **Master Explorer**.
- Can view **Public Folders**.

Review the Basic User and None edit permissions

Basic user

The Basic User edit permission allows you to make simple changes at the user level for yourself, so that without a lot of training, you can optimize forms for your own usage. You cannot create or delete forms, though you can revert to the next most applicable version of forms.

As a Basic user, you can:

- Make basic customizations to your forms such as:
 - Hiding or making a field read-only on a form.
 - Changing the colors and fonts.
 - Rearranging the fields on a form.
- Customize your own **My Folders** folder.
- View **Master Explorer**.
- View **Public Folders**.

None

When you are assigned the None edit permission, you cannot enter **Design Mode**.

With the None permission you can:

- Customize your own **My Folders** folder.
- View **Master Explorer**.
- View **Public Folders**.



For all users, regardless of editing permissions, unless the administrator has disabled this ability, some simple runtime changes can be saved, like grid column sequence and form size and splitter position.



Exercise 2.1: Log in to CloudSuite and assign form editing permission

In this exercise, you will log in to CloudSuite and you will use the Users form to assign a form editing permission.

Exercise steps



Verify you are logged in to the training desktop. If not, log in following instructions provided by your course instructor.

Note: For an SDL course, follow the instructions on the course Lab On Demand screen.

Part 1: Log into CloudSuite

1. Double-click **CSI Click-Once Client** on the desktop. The **Sign In** form opens.
2. Type sa in the **Username** field.

3. Type *Global08* in the **Password** field.
4. Select *CSI_Demo_DALS* from the **Configuration** drop-down list.
5. Click **OK**. The **CloudSuite** application opens.
6. Select **View > User Preference**. The User Preference window opens.
7. Select the **Unload IDO Metadata with Forms** check box.
8. Click **OK**.

Part 2: Create a new user

1. Open the **Users** form.
2. Click **Filter In Place** on the toolbar to turn off the filter-in-place mode.
3. Click **Create a new object in the current collection** on the toolbar. A blank **Users** form displays allowing you to enter data for a new user.
4. Type *<Your Initials> + sitedev* in the **User ID** field. For example: John Smith would type *jssitedev* in the field.
5. Do not enter a password in the **User Password** field. Verify the field is clear.
6. Select the **Super User** check box.
7. Select **Site Developer** from the **Editing Permission** drop-down list.
8. Click the **User Modules** button.
9. Click **Create a new object in the current collection** on the toolbar.
10. Select **Developer** from the Module Name drop-down list.
11. Click **Create a new object in the current collection** on the toolbar.
12. Select **Transactional** from the Module Name drop-down list.
13. Click **Save**.
14. Close the User Modules and the Users form and exit out of the system after saving all modified objects if any.

Part 3: Log Back into CloudSuite and check new user functionality

1. Log back in to the system using your new Site Developer ID as your user ID.
2. Click **Open** to locate a form.
3. Select a form and double-click to open it.
4. Click **Filter In Place**.
5. Click **Design Mode**. The **Identify Your Editing Scope** dialog box opens.
Note: Since you are now a Site Developer and can impersonate other roles in the system, you are prompted to identify your editing scope permission level before you can continue.
6. Select an editing scope from the **Scope** drop-down list.
7. Click **OK**.
8. Close the form.

Check your understanding



Describe the hierarchy of form editing permissions, 1 being the highest and 5 being the lowest.



Description	Hierarchy
Site Developer	
Full User	
Vendor Developer	
Basic User	
None	



The Scope Type and Editing Permissions carry out the same functionality.

- a) True
- b) False



A user with Full Editing permissions can create and delete forms:

- a) True
- b) False



Lesson 3: Editing forms

Estimated time

1 hour

Learning objectives

In this lesson, you will:

- Describe how to change fonts.
- Describe how to create and edit background color.
- Explain how to set a background image.
- Describe how to change basic default properties.
- Explain how to revert to the default version of a form.

Topics

- Change default font
- Set form and component background colors
- Set a background image
- Change some basic component properties
- Revert to the default version of a form

Change default font

In the system, each user selects the theme they want to use, which in turns provides defaults for things like fonts at the form and component level. Themes can also be customized at the site, group, or user level. Themes and the styles they contain are stored in the forms database.

Many style attributes, like form and component fonts, can then be overridden by specific forms. Form-level changes are stored in the forms database.

The **User Preferences** option under the **System** menu before you open a form allows you to change the look and feel of the entire application (the theme), including your forms.

You can also open a form in **Design Mode**. Once you select this option, you can change the font style and size under **Appearance**, located on both the **Form** and **Components** sheets. The **Form** sheet corresponds to the entire form. The **Component** sheet applies to the individual components on the form.

The font can then be further overridden at the component level for these types of components on a form:

- Button
- Check box
- Combo box
- Critical number
- Data view
- Date picker
- Drop list
- Edit
- Grid
- Grid column
- Group box
- Hyper link button
- List box
- Notebook
- Radio button
- Static
- Tree control
- Vertical grid
- Vertical grid element

Changing default fonts using the User Preferences dialog box

To access the **User Preferences** option, go to **System > View > User Preferences** and click the **Run-time Layout** tab on the **User Preferences** dialog box. You will notice that the **Themes** field is set to **Infor**, a default option.

The system has two default themes that you can edit but you cannot delete:

- Infor is the more current of the two themes.
- Classic has the look and feel of previous versions.

Select **Edit** next to the **Themes** field to change the system default theme (Infor). When the **Styles** displays, there will be two panes. The left pane contains a list of the elements, including those within the application which you can edit. The right pane is where you can change font style, size, and color. Depending on the level you select, your edits will impact the entire system and/or its forms.

You will notice in the right pane, there are two sections: Fonts and Colors. You can see that there are different color options listed. For this lesson, we will focus on the Foreground field as this is where you can assign a font color.

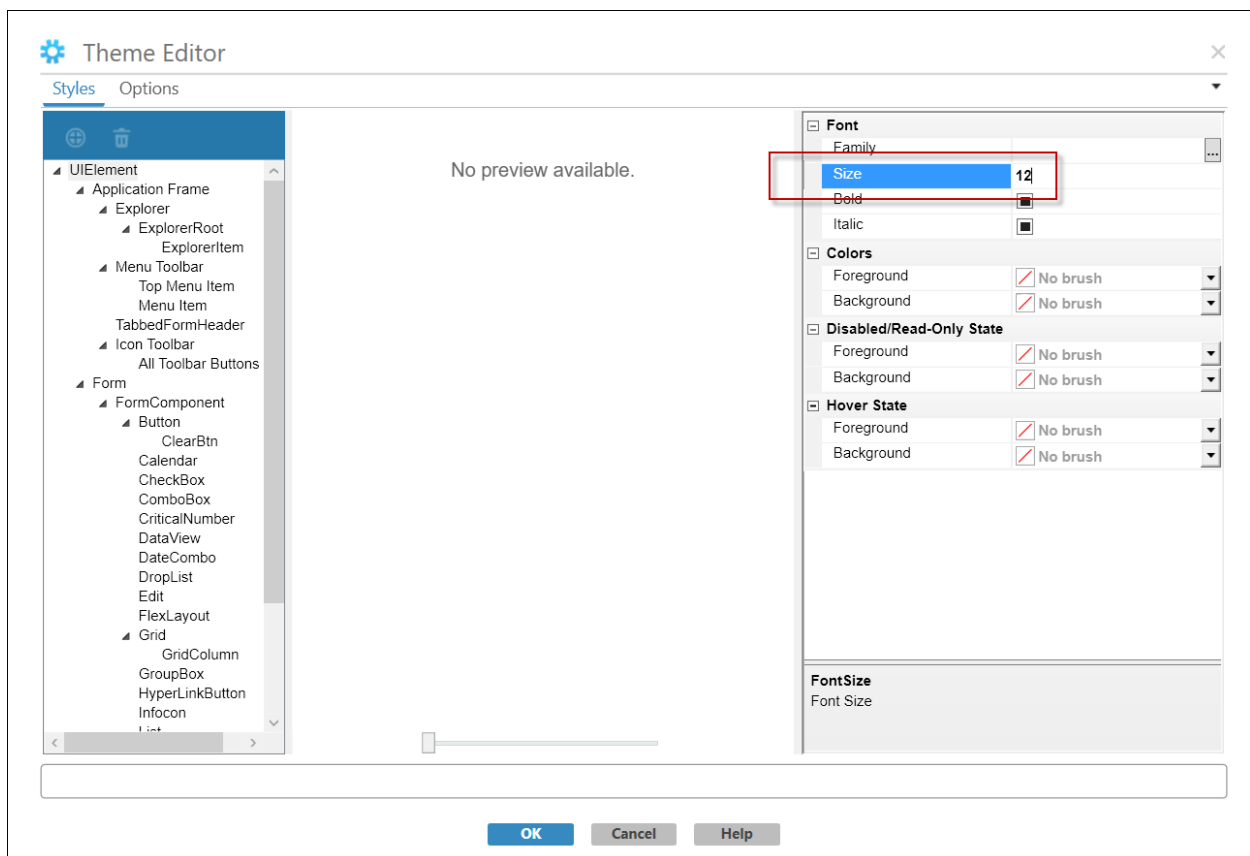
The purposes of the additional fields in the **Colors** section are noted below:

- **Background** – background color
- **Hover Foreground** – text color for a hover box
- **Hover Background** – background color for a hover box
- **Required Foreground** – font color for a required field

- **Required Background** – background color of a required field
- **Disabled Foreground** – font color for disabled field
- **Disabled Background** – background color for a disabled field
- **TBD Foreground** – font color of field that is automatically filled in by the system if the user does not provide a value
- **TBD Background** – background color of a field that is automatically filled in by the system if the user does not provide a value

Since you want to change the font style, sizes, and color for forms, select either Form or a component element in the left pane. Then in the right pane in the **Font** section, type the **Font Family** name, **Size**, and select **Bold** and/or **Italic** or **Italic**, as applicable.

To change the font color, you would click in the **Foreground Color** field and indicate the color of the font. If you chose to format a component, it will appear in the preview pane. Click **OK** to save your settings and go back to the main screen.



Example of default font changed in User Preference

Now you can open a form to view your updated formatting.



You can also access the **Themes** dialog box by opening a form, going into **Design Mode**, and selecting **Edit > Theme**.

Using the form and component sheets to change default fonts

You can change the default fonts for your forms and components by going to the **Form** and **Components** properties.

The following is a high-level overview of how to change the default forms on your forms and components.

Begin by opening form. Then, click **Filter In Place** to remove the filter and display all the fields in the form. Next, you click **Design Mode** and confirm your editing scope, if applicable. The form will be displayed in design mode.

There are two panes displayed, one each side of the form. We will be using the **Toolbox** in the left pane to add new components to the form later in the training. For this lesson, you will use the **Form** sheet to modify the attributes of the form. The components on the form are modified in the **Component** sheet. To access the **Component** sheet, you can either click on a component within the form itself or click on the **Component** link below the properties window.

You will notice in addition to the **Form** and **Component** properties, there is also an **Objects** link under the left pane. The **Objects** sheet provides you with options to work with specific global objects. You can use this sheet to work with global objects independent of their associations with forms. We will be focusing on the **Form** and **Component** sheets to change the default font settings on a form.

To change the characteristics of the font for the entire form using the **Form** sheet, confirm the form name is displayed in the **Name** field under **Design**. Then expand the **Appearance** section to change the font color in the **Foreground** field. You can choose the font color by clicking the arrow in this field and selecting a color. Next, click the ellipsis (...) button in the **Font** field to select a font style and size. You can see the changes on the form by clicking the mouse on the form, away from the fields.

Regenerating a form

In some cases, you will be able to see your editing changes immediately. However, there are instances when your changes will not immediately be noticeable. To see those changes, you will need to regenerate the form. When you regenerate a form, the system saves your changes, then closes and reopens the form in **Design Mode**.

There are two ways to access the **Regenerate Form** option:

- Click the **Regenerate Form** icon on the toolbar, or
- Select **System > Edit > Regenerate Form** using the menu.

You can exit **Design Mode** to see the newly-formatted form as an end user will see it. If you need to make additional changes to the form and/or its components, you will need to go into **Design Mode** again.



You can do these steps one at a time by saving the form, closing it, and re-opening it, or regenerate it in one step.

Set form and component background colors

We have learned how to change the default font and set a background image on a form, now let's look at how we can change background color on forms and components.

The primary settings for both forms and form components come from the system default settings. Background colors can either be:

- A single, solid color, with or without transparency, or
- A pair of colors used for a linear gradient; each color with its own RGB and transparency settings.

To override the default background colors for forms and form components, you can either use the **Edit Color** dialog box or the **Form** and **Component** tabs. You are familiar with using the **Form** and **Component** tabs to change font attributes, here you will use them to define the background color for the form and/or its components.

You may have noticed that some of the fields on the **Form** sheet have settings that are already populated. The background color field is one of the fields that will display a value that can help you to understand the color being used in the form. When you have a form opened in **Design Mode**, you can look on the **Form** or **Component sheet > Appearance > Background Color** to see if one of the values below is displayed as the current value for the form background.

If the Background Color field display is...	It means that...
Blank	No background color has been set.
A red, green, blue (RGB) value (such as 12, 127, 242)	A single, solid color has been set using the Color attribute.
V(bgc_Form)	The background color was set for the form using the Background Color attribute settings on the form property sheet.
V(bgc_componentName)	The background color was set for a component, using the Background Color attribute settings on the component property sheet.
V(gbgc_colorDefName)	The background color was set using a global background color definition variable created in the Edit Color dialog box. In this case, colorDefName is the name assigned to the color definition variable in the Enter a Name for the Background Color dialog box when the definition is created or updated.

Using the form and component properties to set background color

To set the background color on a form or component using the **Form** and **Component** properties, you can choose to use an existing color by clicking the ellipsis (...) button next to the **Background Color** field on the **Form** or **Component** sheet. To edit or add background color for a form, go to the **Form** sheet and select background color from the **Background Color** field. To edit or add background color for a component, you would use the **Component** sheet.

You can choose a solid color by selecting the arrow next to the **Color** field. The value of the color is expressed using standard RGB definitions. You can also type the RGB values directly in the field separated by commas.

If you want to apply a transparency to the solid color, use the extended color attributes. The **Extended Color Type** field can be used to designate if your chosen solid color should be set to:

- **None** – Indicates that no extended color settings are being used. It is the default setting. If you keep this option it disables all the other extended color attributes.
- **Color** – Indicates that a single (extended) color is being applied, and that transparency settings are also available. Because it is a single color, only the **Ext. Color 0** settings are used.
- **Gradient** – Indicates that a linear gradient is being applied, and that each extended color has its own possible transparency setting. **Ext. Color 0** is the starting color, and **Ext. Color 1** is the ending color.



When using a gradient pair, be aware that you cannot specify **Ext. Color 1** until both the **Ext. Color Type** attribute and the **Ext. Color 0** attribute have been set. The advantage of using **Ext. Color 0** is that you can also use a transparency setting with it.

Using the Edit Color dialog box to set background color

The **Edit Color** dialog box allows you to create and/or apply a background color which you can later reuse for multiple forms and components.

To set the background color for a form or component using the **Edit Color** dialog box, open the form you want to change and click **Design Mode**. Then go to **System > Edit > Background Color** to display the Edit Color dialog box. In the **Edit Color** dialog box, you will have the option to apply an existing color, create a new color, or delete an existing color. To apply a color, simply select it from the dialog box and click **OK**.

After you have applied color settings to your form or forms, you may have to regenerate it or close it and re-open it for the changes to take place.

The example below shows a form with the font style, size, and color changed. The background color on the panel and on the form itself was changed. You will have the opportunity to practice formatting a form at the end of this lesson.

Users

User Name: ✓ Super User

User Description:

Workstation Domain/ID:

User Password:

Confirm Password:

Editing Permissions: Site Developer User Modules Row Authorizations... User Authorizations...

Groups Login Information Email Address Source Control

Group Name	Group Description	Primary Group
★ 1 ▶		☐
✱		

Group Authorizations... Row Authorizations... Copy Groups From User

*Example of formatted **Users** form*

Resetting a form to its previous format will be covered later in this lesson. However, you can choose to reset the background color of a form to the system defaults by clicking the ellipsis (...) button to the right of the **Color** drop-down box.

Set a background image

You can add a background image to a form by using the **Background Image** form property. By using this property, you can specify a graphics file to use as the background image on a form and how that image is to be displayed.

The **Background Image** form property has four associated attributes:

- **Image**
- **Center**
- **Disable On Pane 0**
 - When set to **True**, this attribute excludes the image from pane 0.
 - When set to **False**, this attribute includes the image on pane 0.
- **Disable On Pane 1**
 - When set to **True**, this attribute excludes the image from pane 1.
 - When set to **False**, this attribute includes the image on pane 1.

The **Image** attribute of the **Background Image** property is used to specify the file name of a graphic image to be used as a background for a form.

The system supports the following graphics formats:

- | | |
|--------|--------|
| • BMP | • ICO |
| • EMF | • JPG |
| • EXIF | • PNG |
| • GIF | • TIFF |
| • WMF | |

Importing background images into the system

To use an image as a background on a form, the image must be able to be located by the system. If you specify a path as well as the image file name, this path must be valid for every user accessing the system. Therefore, it is important that the file resides within the application.

You can use the **Images** dialog box to import images to the forms database. This makes it possible to transport the image file with the form and eliminates the need to call the image from an external location. To access the **Images** import dialog box, enter Design Mode, and select **System > Edit > Image**.

Setting a background image on a form

You can add a background image to a form by first opening the form, selecting **Design Mode**, and setting the image attributes in the **Form** properties sheet. You simply scroll down the different attributes on the **Form** sheet to the **Background Image** section. In the **Image** row, you can select an image you've imported to the forms database from the drop-down list.

Once you select the image, it is automatically scaled to fill the form by default. However, you can choose how the image displays or aligns by applying additional attributes.

Setting a banner

You can set a banner to a form by using the Static component and selecting an image file in the **Bitmap File Name** field. You can also specify how the image displays under the **Data Source > Formatting > Image Display** property and set specific attributes for the image, including its size.

Change some basic component properties

As you have seen from the previous lessons, the system gives you many options to personalize forms and their components. Now we are going to take a closer look at the other properties that can be changed on a form.

Let's suppose that you have created a form and you want to control the behavior of the components on the form. For example, you may want to hide a field or set it to read-only. The system allows you to set behavior properties for components.

You have already learned how to use the **Component** sheet to change font properties and to set background color on components. The **Component** sheet has many attributes divided into sections. One section is Behavior, with attributes like Hidden or Read Only. You can change a component to Read Only by changing this attribute to True. Read Only components will not allow the user to change values for types like Edit or ComboBox, or click them for types like PushButton.

Setting a field to read-only

You can set a component to read-only so that it does not respond to end user interaction. When you set a field to read-only, you are not allowing the end user to edit the data displayed in the field. The read-only behavior is automatically assigned to any component that corresponds to that read-only property regardless of the value of the component's **Read-Only/Disable** property. You can make a component that is associated to read-write component read-only by setting that component's property to read-only.

The following is high-level overview of how to set a component to read-only.

Begin by opening the form in **Design Mode**. Then, you click the appropriate component. The **Components** sheet will display.

Next, you scroll to the **Behavior** properties and expand the menu. You click the down arrow in the **Read Only/Disable** field and set the value to **True** for the selected component.

Hiding a component or field

You can also use the **Behavior** properties to control if a component or a field is visible to the end user.

The following is a high-level overview of how to hide a component or field.

Begin by opening a form in **Design Mode**. Then, you click the component and locate the **Behavior** properties on the displayed **Component** tab. Finally, click the drop-down arrow in the **Hidden** field and select **True**. The component will now be hidden.

You may need to regenerate the form or save it then reopen it to view your changes. You may also have to deselect **Design Mode** to see the hidden components.



You can also use the **Visible When** property to make components visible or hidden based on certain conditions.

There are additional behaviors that you can apply to your components. The table on the next page explains the purpose of these behaviors.

Behavior	Purpose
No Tab Stop	Use the No Tab Stop property to omit a component from tab order.
Auto Complete property	The Auto Complete property applies to components with a list source, such as combo boxes. It can be used to display a list when the user begins typing. The setting description: • Set to True to display the list when the user begins typing. • Set to False (the default) to display the list only when the user clicks the drop-down button.
Add/Details and Find form	Use Add/Details to specify the name of a form that is opened when the user selects Add or Details from a component's right-click menu. Use Find to specify the name of the form that should open when the user selects Find from a component's right-click menu.
New and Copy operations	Use this attribute to specify whether the user can create new rows in a collection and/or make copies of existing rows. Creating a new row is a WinStudio standard operation and applies to primary and secondary collections.

Setting a field to be dynamically disabled or enabled

Another interesting behavior that is available is the ability to dynamically disable or enable a field. You can use this feature to assign behaviors to components and fields. The system performs certain operations or displays certain behaviors based on the identified conditions.

You set one or more of the properties in the **Dynamic Enabling and Disabling** fields. You have the option to apply dynamic enabling and disabling behaviors to your forms using the following fields:

- **Right-Click Menu** – Use this property to specify the event that occurs when the right-click action is used.
- **Drag Image** – Use this property to set an image to display when a dragging action occurs. You may select an existing image or add a new one.
- **Enabled When** – Use this property to make components enabled or disabled based on certain conditions.
- **Required When** – Use this property to specify the conditions under which a value for this component is required.
- **Visible When** – Use this property to specify the conditions under which this component is visible.

To apply the behavior, either select the appropriate event from the drop-down list or click the ellipsis (...) button next to the property field to set the specific conditions. You may indicate the condition in the **Edit Condition** dialog box. You may select an existing event or create a new one.

Using images in components

You can display images in static components that have no data binding, in buttons, and in tree components. The images that you use must be imported into the forms database by entering design mode, and selecting **System > Edit > Image** so that when you distribute your form, the system can locate the image you used.

The following is a high-level overview of how to display an image in a component.

Begin by opening your form in **Design Mode**. Then, select a component. In the **Component** property sheet, specify a graphic file name for the **Bitmap File Name** property. The image will be displayed in the component field.



If you specify a path as well as the image file name, this path must be valid for every user accessing your application.

You can also add an image to a newly-created static or button component. We will cover how to create new components in the next lesson.

Formatting date, time, and other specific data in components

The default formats for date and time are those specified in the regional settings in the Windows Control Panel. You can override date, time, and other specific data formats. These settings can be inherited or overridden at any point in the inheritance hierarchy.

The properties and descriptions for specific data formats are:

- Date Format
 - Specify whether date and time data include the date only, the time only, or both the date and time.
- Default IME Character Set
 - Set the Input Method Editor (IME) to a specified character set. This option is used only with components that receive input from Asian language keyboards.
- Binary Data
 - Specify the file extension of binary data. This option is used only with components bound to a property that contains binary data.

Formatting input masks in components

You can specify a mask or template to apply when text is entered in a component (such as a phone number). Input masks determine the format the system should use to display values in that field.

The properties and descriptions for input masks are:

- Input Mask
 - The pattern for the mask. You can select from the default patterns or enter your own. Patterns include a separator character.
- Prompt Character
 - The character used in the pattern to signify empty spaces



When you specify an input mask, you should consider whether a null value is allowed in that field. To prevent the system from attempting to save an unspecified value in a decimal field bound to a table column that does not allow null values, use the mask character 9. An input mask that contains 9 defaults to 0 if no value is specified in the field and the value 0 can be saved.

Revert to the default version of a form

The system helps you to determine if a form has been customized. You can locate information regarding a form including if it has been customized by navigating **System > Help > About This Form**. If the form version is not "Vendor Default" then the form has been customized.

This is important information to know if you don't want to use the customized version and want to revert the form to a previous version. You can revert a customized form to a previous version by using the **Revert Form Definition** menu option.

With Site Developer, Full User, or Basic User editing permissions, you can cancel customizations to a form and any global form objects associated with the form, and restore it to another version per this hierarchy:

- A "Site Default" version reverts to the default version.
 - A "Group" version reverts to a site version (if one exists).
 - A "User" version reverts to a group version (if one exists).



You must have a form open and be in **Design Mode** to revert a form.

Copying a form

You may want to make a copy of a form before you revert it in case you want to use the customizations later. With the Site Developer editing permissions, you can copy the customized form to a new form name before you revert it. Then, after reverting the existing form, you can copy any customizations desired to a new standard form and customize as needed.

The following is a high-level overview of how to make a copy of a form.

Begin by first opening the form and being in **Design Mode**. Then, select **System > Form > Definition > Copy**. The **Copy Form Definition** dialog box will display.

You enter the name of the form being copied in the **Source Form** field and the name of the new form in the **Target Form** field.



To copy only the form definition, do not select the **Copy Template Also** check box. If you want to copy the source form's template as well as its definition, select the check box.

Finally, you leave the check box checked under **Device Types** and click **OK**.

Reverting a form

The following is a high-level overview of how to revert a form.

You must first open the form and be in **Design Mode**. Then, select **System > Edit > Revert Form Definition**.



You can also access this option by clicking **Design Mode** and then selecting **Edit > Revert Form Definition**.

You click **OK** to confirm the request when prompted and close the form and select **Form > Definition > Unload All Global Form Objects**.



When you close a form, the system keeps the form and any associated global objects in its memory. As a Site Developer, if you change your editing scope by impersonating a different user, you must clear these forms and global objects from memory to display the versions that correspond to your new editing scope.



Demo: Change font on form

This demonstration shows how to change the font on the Users form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 3.1: Change font on form

In this exercise, you will change the font on the Users form.

Before you begin:

- Ensure you have completed Exercise 2.1 because it provides data or configurations for this exercise.

Exercise steps

1. Double-click the **system** icon on the desktop.
2. Log in using the information provided by the instructor. The **Infor CloudSuite** application opens.
3. Click **Open** on the toolbar.
4. Locate the **Users** form from the list.
5. Double-click the form name to open it.

6. Click **Filter In Place** on the toolbar to turn off the filter-in-place mode.
7. Enter **Design-Mode**.
8. Click on the **User Description** static field.
9. Click + to expand the **Appearance** section in the **Form** property tab.
10. Click the arrow in the **Foreground** field and use the color picker to choose a font color.
11. Click the **ellipsis** in the **Font** field to select a font style and size.



Demo: Add background color to form

This demonstration shows how to add a background color to the Users form and/or its components.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 3.2: Add background color to form

In this exercise, you will add a background color to the Users form and/or its components. You will also regenerate the form and then revert it to its previous version.

Before you begin:

- Ensure you have completed Exercise 3.1 because it provides data or configurations for this exercise.

Exercise steps

1. Select the **Form** sheet to change the color on the entire form or click the component to which you want to apply the background color definition.
2. Click **Background** under the **Appearance** section to expand the properties.
3. Click the down arrow in the **Color** dialog box.
4. Select a color for the background.
5. Click out of the box to see your chosen background color.
6. Select **System > Edit > Regenerate Form**. The form is refreshed.

7. Select **System > Edit > Revert Form Definition**. The form is reverted to its previous version.



Demo: Set a background image

This demonstration shows how to set a background image.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 3.3: Set a background image

In this exercise, you will add a background image on the Users form.

Before you begin:

- Ensure you have completed Exercise 3.2 because it provides data or configurations for this exercise.

Exercise steps

1. Open a form to set a background image.
2. Click **Design Mode** on the toolbar menu, to display the **Form** sheet for the form.
3. Double-click **Appearance** in the **Form** property sheet. The menu expands to display options.
4. Scroll to **Background Image**.
5. Click **Background Image** to expand the menu.
6. Select **Radar.png** from the **Image** drop-down list.
7. Select **True** from the **Center** drop-down list.
8. Click **Behavior > Initial Command to Refresh**.
9. Click **System > Edit > Regenerate Form**. The form is refreshed and your selected background image is displayed.

Note: You can also save and close the form. Then reopen it to see the added image.

Check your understanding



Explain how to revert to the previous version of a form.





To use an image on a form, the image needs to be _____ into the system:

- a) Imported.
- b) Exported.
- c) None of the above



To change the properties of a component on a form, you must use the:

- a) Component Sheet/Tab.
- b) Form Sheet/Tab.
- c) Object Sheet/Tab.
- d) None of the above



Lesson 4: Customizing form components

Estimated time

1 hour

Learning objectives

After completing this lesson, you will be able to:

- Identify different types of components in the system forms.
- Explain how to add components to forms.
- Explain how to manipulate the alignment, size, and location of a component on a form.
- Identify the steps to set the tab order of components.

Topics

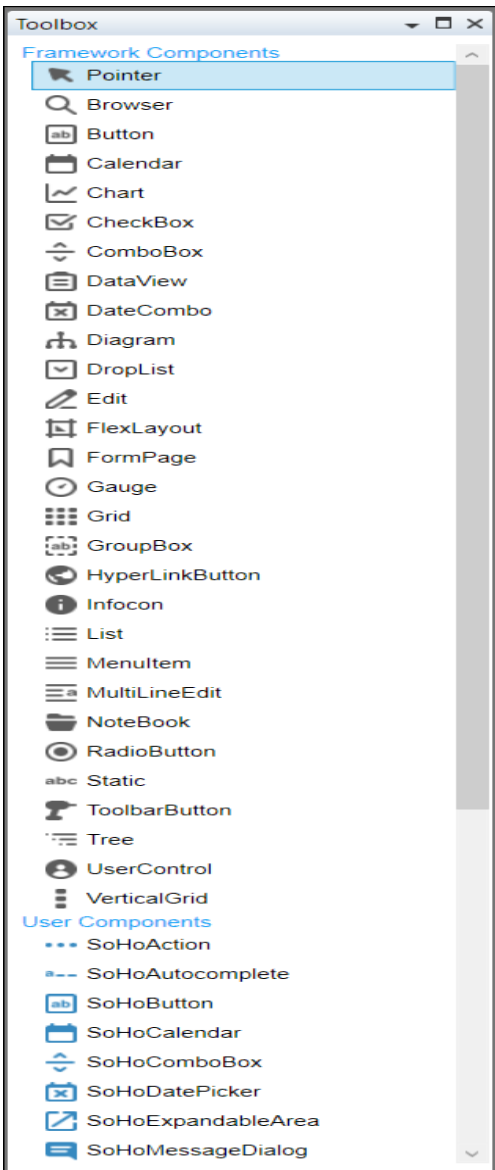
- Explore the system components
- Add components to a form
- Set up components on a form
- Set tab order for components

Explore the system components

The system provides you with a wide variety of components that you can use to present and manipulate data in your forms. You can select and add these components to your forms by using the component **Toolbox**. The **Toolbox** contains all the available components that you can use on your forms.

In this lesson, you will learn about the components available in the system and how to use the **Toolbox** to select and add components to your forms. First let's look at some of the components available for your use.

The table below describes each of the currently supported component types and the most commonly used components.

Components in Toolbox	Most commonly used components
 The screenshot shows the Toolbox window with two sections: Framework Components and User Components. Framework Components include Pointer, Browser, Button, Calendar, Chart, CheckBox, ComboBox, DataView, DateCombo, Diagram, DropDownList, Edit, FlexLayout, FormPage, Gauge, Grid, GroupBox, HyperLinkButton, Infocon, List, MenuItem, MultiLineEdit, Notebook, RadioButton, Static, ToolbarButton, Tree, UserControl, and VerticalGrid. User Components include SoHoAction, SoHoAutocomplete, SoHoButton, SoHoCalendar, SoHoComboBox, SoHoDatePicker, SoHoExpandableArea, and SoHoMessageDialog.	Button – Used to initiate an action such as running a procedure or launching another form.
	CheckBox – Used to display logical selection options between two choices such as yes/no, true/false, on/off, etc.
	DateCombo – Used to generate a calendar for user selection.
	Edit – Used for the display and input of text via standard keyboards. This field can only display a single line. For multiple lines of text, you must use the MultiLineEdit component.
	Combo – Used to allow the user to type text directly into the component or to select from a drop-down list of values.
	Grid – Used to allow data to be displayed and maintained in a tabular format.
	GridColumn – An individual column in a grid component. Grid columns can act like edit, combo box, or check box components. It can only be created using the Edit Contained Components dialog box.
	GroupBox – Used to frame a group of related components.
	MenuItem – Used to create menu options that are available from the Actions menu
	NoteBook – Used to group components together on tabs to manage screen space.
	NotebookTab – A tab of a notebook. It can only be created using the Edit Contained Components dialog box.

Components in Toolbox	Most commonly used components
	RadioButton – Used in groups with each button representing a member of a set of mutually exclusive options. The user can select one option.
	Static – Used to display read-only text such as field labels. This component can also be used to display graphics images.
	ToolBarButton – Used to create toolbar buttons.

The following components are either not used at all or are used very rarely within the system application:



- **Browser**
- **DropList**
- **FormPage**
- **Graph**
- **HyperLinkButton**
- **List**
- **MultiLineEdit**

The **Tree** and **User Control** components are complex and beyond the scope of the content covered in this course.

Accessing and using the toolbox

You can access the **Toolbox**, which is docked on the left side of your screen, while in **Design Mode**. The **Toolbox** displays as a default setting when a form is opened in **Design Mode**. However, you can choose to hide the **Toolbox** and **Object Viewer** which are both on the left side of the screen by deselecting them under **System > View**.

Add components to a form

Now that you have learned about the system components and the **Toolbox** where they are found, let's look at how you would add components to a form.

The process of adding, arranging, and aligning components on a form is the same whether you are working with an existing form or creating a new form. Remember, anyone with Full User or Site Developer permission can add components to forms. If you are a Full User your changes will only affect your user-level forms.

The following is a high-level overview of the simplest way to add a component to your form.

Begin by opening the form in **Design Mode**. Then, click and drag a component on the **Toolbox** list. You move your mouse to the form. The mouse pointer will change to crosshairs.

When you drag your mouse to draw the component to the size you want, you will then see the **Component** sheet display.



You can also click the form to draw the component at its default size.

Finally, click the **Component** sheet to assign properties, characteristics, events, and attributes to the component.

If you change your mind about adding the component, you can always delete it. Select it and press **Delete** on your keyboard. You can also go to **System > Edit > Component > Delete Selected**.

Set up components on a form

Now that you have learned how to add components to a form, we will discuss some of the options that are available to help you manipulate components within the form.

Below are the steps to help you set up your components. These instructions assume that you have your form opened in **Design Mode**.

Sizing a component

You will probably want to size a component once you have added it to a form.

The following is a high-level overview of how to size a component.

Begin by clicking the component you wish to resize. Then, pass the pointer over one of the black handles until it turns into a two-way arrow. You click and drag the handle to the new size.

Aligning the components

There are two options to position components in the system. You can drag them into position or you can use the **Alignment** toolbar, which is also referred to as the **Design Mode** toolbar.

You can align components individually or in a group by selecting one component or a group of components and then using your mouse to drag the component or components to the desired location on your form.

- The following is a high-level overview of how to align one or more components.
- First, you click once on the component or use control-click to select a group of components to align. Next, you release your mouse once you have dragged the component or components to the desired position.
- You can also use the icons on the **Alignment** toolbar to help you position more than one component. The **Alignment** toolbar is located under the **Main** toolbar. It is only accessible in **Design Mode** and becomes active when you select two or more components.
- Once you select the components using control-click, click any of the commands below to align them to each other.

Command	Description
Align Left	Left align the components
Align Right	Right align the components
Align Top	Top align components
Align Bottom	Bottom align components
Align Center	Horizontal center alignment
Align Middle	Vertical middle alignment
Make Same Width, Make Same Height, or Make Same Size	Resize components to match the specific sizing of the target component

Command	Description
Space Horizontally	Evenly space components within the horizontal boundaries
Space Vertically	Evenly space components within the horizontal boundaries
Arrange in a Column	Align components into one column
Arrange in a Row	Align components in a row
Bring to Front	Positions the selected components to the front, placing them on top of other, overlapping components
Sent to Back	Sends the selected components to the back, placing them underneath other, overlapping components
Undo	Undo the previous design action; to undo multiple actions, click this button repeatedly
Redo	Repeat a design action that has been done; to redo multiple actions, click this button repeatedly

These alignment options align selected components with whichever component has solid black squares for handles. After you select all the components you want to align, click a component to align with, and all the components will be aligned. The handles of the last component you selected should be solid black, and all others should be black-outlined handles.

Set tab order for components

Users can press the **Tab** key on their keyboard to move between components on a form. You can determine how you want a user to tab through the components on your form. This can be especially important when you are adding or removing components on a form.

Using Tab Order to set tabs for components

The **Tab Order** on the **Alignment** toolbar allows you to set the order of tabs for the components on a form. This option resets only the components starting with the first one you select, which is zero-based. The sequence of the components to that point does not change.

Following is a high-level overview of how to set the tab order of all components on a form, beginning with the first (0) component.

You open a form and go into **Design Mode**. Then, perform one of these actions:

1. Click System > Edit > Component > Tab Order.
2. Click Tab Order on the Alignment toolbar.
3. Click the components on the form in the order in which you want users to move to them with the **Tab** key.



A tag near the upper-left corner of each component indicates the sequence number of the component in the tab order.

4. Save the form, then close and re-open your form to see the tab sequence changes.

When making your selection, be sure to select:

- Every component that you want to include in the tab order.
- A static component used as a label for a component immediately before you select its companion component.
- A container before you select the components contained within it.

To exit tab order mode, do one of the following:

- a. Click **System > Edit > Component > Tab Order**, or
- b. Click **Tab Order** on the **Alignment** toolbar.



You can set the tab order on only one form at a time. You should exit tab order mode for the current form before you attempt to set the tab order on another form.

Omitting a component from tab order

If you want to omit a component from a tab order, you can use the **No Tab Stop** property in the **Component** sheet. The following is a high-level overview of how to set a component to **No Tab Stop** in the **Component** sheet.

First, go to **Behavior**. Then, select **True** from the **No Tab Stop** drop-down list.

Identifying tab order tags

As the tab order is set for components, their color is changed to show the state of components in the tab order session.

- Blue text on white indicates that the component has been clicked.
- White text on red indicates that the component is the last component clicked.
- White text on blue indicates that the component has not yet been clicked.



On extended forms, dark blue text on a light blue background indicates that the component belongs to the base form and you cannot change the tab order.



Demo: Add components to a form

This demonstration shows how to add components to a form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Scenario

Progressive Cycles sent out a service person to perform maintenance on a couple of our bicycles that were shipped to our customer H.H. Squash. You need to update the Customer Orders form in the system.



Exercise 4.1: Add components to a form

In this exercise, you will add some components to a form. Then, you will arrange and align them with each other or existing form components.

Before you begin:

- Ensure you have completed Exercise 3.3 because it provides data or configurations for this exercise.

Exercise steps

1. Open the **Customer Orders** form in the system.
2. Remove the filter-in-place, if applicable.
3. Enter **Design Mode**.
4. Select the **User Defined** tab.
5. Add **ComboBox** to your form.
6. Add two **Edit** boxes and two **Static** boxes to your form.
7. Add two **Buttons** to your form.
8. Align your **Static** and **Edit** boxes next to each other. They should be the same height.
9. Align the **ComboBox** to the right of your first **Edit** box.
10. Place your buttons in the middle toward the bottom of your form. They should be the same height.
11. Add one more **Static** component to your form.
12. Align and resize this component with your **ComboBox**. They should be the same height.
13. Click the 'Edit notebook tab(s)' icon,  on the top left of the **Notebook** tab.
14. Click the **UserDefinedTab** under the Items window.
15. Expand the **Behavior** node.
16. Select **False** for the **Read-Only/Disable** field.
17. Click OK.
18. Save and regenerate the form.

Check your understanding



Name some of the components that can be added to a form.





Describe how you would add a component to a form.





Name a tool that is used to align components on a form.





Lesson 5: Using inherited attributes

Estimated time

.5 hours

Learning objectives

In this lesson, you will:

- Explain the value of inherited attributes.
- Describe how to use the inherited attributes.
- Explain how property class extensions are applied to a component.
- Describe how to bind components to an IDO property.

Topics

- Understand the value of inherited attributes
- Use component classes
- Use property class extensions
- Bind components to an IDO property

Understand the value of inherited attributes

Now that you have learned about formatting and setting up components on a form, in this lesson you will see how you can save time assigning attributes to components by using inherited attributes in the system.

In the system, there is an IDO property class called Item which has attributes like default label \$Item. These attributes are inherited by all IDO properties that are of this datatype. Components on forms bound to these properties also inherit these attributes. These attributes can be overridden at multiple levels. Overrides should be minimized to simplify maintaining these attributes.

In addition, you learned:

- The three-tier architecture in the system is like a car manufacturing process.
- There are many reusable components in the system.
- To save time, you should make sure you utilize pre-existing work.
- When you use inherited attributes, you are reusing components rather than duplicating them and this will help you to save time and significantly reduce the size of the application.

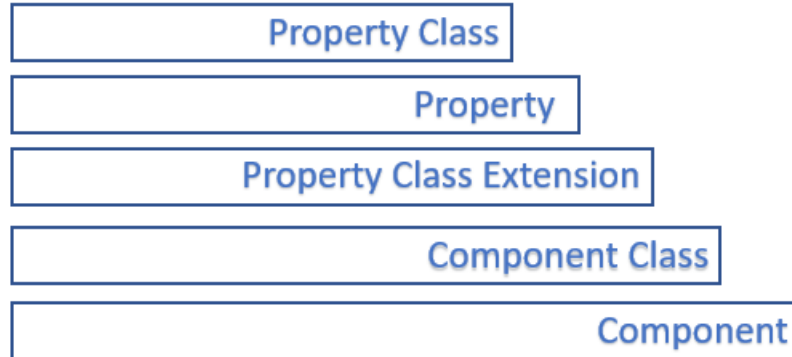
Since it is very common in most applications for the same attributes to be referenced many times throughout the application, you can use the principal of inheritance in the system to set up inherited behaviors for your components.

1. Let's review the terms associated with inheritance attributes in the system.

Term	Description
Property class	• An IDO property class is created at the IDO level. • When a property is created, you can choose to have it inherit the property class' defaults (length, data type, etc.). • Once a property class is used to create a property, you can override any of the defaults on the property. <i>Properties override property class defaults.</i>
Property	• A named, descriptive piece of information. • When a property is used with all other properties of an IDO, it makes up the collection of data that is passed between the client and the database.
Property class extension	• A type of global object that overrides the attributes of an IDO property class of the same name. • Property class extensions apply only to components that are bound to IDO properties for which there is an IDO property class. • It lets you standardize how components override the IDO property class behavior. • A component can override the attributes of its property class extension. <i>Property class extensions override IDO property defaults.</i>
Component class	• A type of global object that lets you reuse sets of component attributes by packaging them together.

Term	Description
	○ For example, instead of specifying a caption, a list of values, and a validator for each component, you can create a component class that bundles each of those attributes. ○ You can then base a new component on the component class to inherit those attributes automatically. ● Updating a component class automatically updates every component that inherits from the class. A component can override the attributes of its component class.
Component	● The actual object that you can create and/or manipulate on a form such as a textbox, a combo box. ● You can manually associate a component with a component class and/or a property class extension. ● You can override either of these defaults by setting individual properties for the component. ● Setting specific properties on a component does not make the component reusable. Components properties override everything.

2. The levels of inheritance for components bound to IDO properties are:



The inheritance architecture for components bound to IDO properties

3. Components not bound to IDO properties have two levels of inheritance which are:



The inheritance architecture for components not bound to IDO properties

If you follow the following rules, you should rarely have to specify a data type, a format definition, and many other properties of a component.

Establish IDO property attributes that are inherited effectively by many components without being overridden when you develop an IDO.

- You can set up IDO property classes to package default settings for property attributes.
 - All IDO properties based on an IDO property class inherit the class attributes.
4. Set up property class extensions only if you need them when you develop forms.
 - If components bound to IDO properties commonly override the attributes of the IDO property and if the IDO property inherits from an IDO property class, you can set up a property class extension to standardize how these bound components override the IDO property class behavior.
 5. Set up component classes to package the attributes of any type of component when you develop forms.
 - Modifying a component class is a streamlined way of altering the behavior of any component that inherits from the class.

You will see in the next section how you can set up and override inherited properties in the system. You will be given selection options on the component properties tabs which are directly related to the ability to use inherited properties.

Use component classes

You should now understand how inherited attributes work and the inheritance hierarchy in the system. In this section, you will learn how to use inherited attributes in component classes to package the attributes of any type of component. When you modify a component class you are streamlining a way of altering the behavior of any component that inherits from the class.

We will begin this section by learning how to create a component class and then we will discuss how to apply inherited properties to a component.

Creating component classes

You can create a component class for any type of component. The packaged attributes are inheritable in the inheritance hierarchy.

The following are inheritable attributes:

- General attributes such as captions, tool tips, and read-only or disabled behavior
- Data attributes such as default values, data types, list source, and validation
- Display attributes such as justification, font and color, and formatting for numbers and special types of data
- Behavior attributes such as shortcut menus, event generation, drill-down to **Find** or **Add/Details** forms, and attributes for dynamically enabling or disabling a component
- Other attributes which are specific to a type of component

You use the **Component Class** dialog box to create, copy, edit, or delete component classes or property class extensions. The dialog box contains a list of all component classes or all property class extensions that are stored in the forms database as global objects. You don't need to have a form open to access this dialog box.

The following is a high-level overview of how to create or edit a component class.

Begin by entering **Design Mode**. Then, select **System > Edit > Component Class**. The **Component Class** dialog box will display.

Next, select an existing component class from the list and click **Edit** or click **New** to create a new component class. Now, you can edit the existing properties or specify the new attributes for the component class and click **OK**.

You can also delete an existing component class or copy and edit an existing component class.

Applying inherited attributes to a component

The following is a high-level overview of how to apply the inherited attributes to a component.

Begin by opening a form in **Design Mode**. Then, click an existing component or add a new component on the form. The **Component** sheet will display. Next, you click **Inheritance** properties to expand the section. Finally, select an existing component class from the drop-down list in the **Component Class** field.



If you want to apply specific component class properties to a component that is not to be inherited by other components, you can do so by clicking the **ellipsis** in the **Component Class** field. Once the **Component Class** properties box displays, you specify the attributes for the component and then click **OK** to return to the form screen.

Under the **Inheritance** properties, you will also notice the **Component Class Parameters** field. This field is used to specify the parameters of the **Component Class** that you selected. For example, you can set parameters like validation, list sources, event handlers, and other properties for the component.

The following is a high-level overview of how to launch the **Edit list of Component Class Parameters** dialog box.

Begin by clicking the **ellipsis** in the **Component Class Parameters** field. Then, perform one of the following tasks:

1. Select **Edit** to change an existing parameter for a component class.
2. Click **Delete** to remove an existing parameter.
3. Click **Create** to set up new parameters for the selected component class.
4. Click **Up** or **Down** to change the order of the selected parameter.

Once you performed the task, you can click **OK** to return to the form screen.

There are rare instances where you may wish to override the existing **Component Class** properties with those you have added to your component. To do so click the ellipsis (...) button in the **Set Class from This Component's Attributes** field.



This action is not recommended because it will overwrite the existing component class attributes with those of your current component, which may be shared by many other components.

Use property class extensions

Property class extensions can override attributes inherited from the IDO property, and add some user-interface attributes not defined on the IDO property. Property class extensions are automatically used by components bound to IDO properties that are derived from a property class. If a property class extension exists for the class of the property the component is bound to, it will be automatically applied.

You will find a list of all property class extensions that are stored in the forms database as global objects in the **Property Class Extension** dialog box.

The attributes that are inheritable are:

- General attributes such as captions, tool tips, and read-only or disabled behavior
- Data attributes such as default values, data types, list source, and validation
- Display attributes such as justification, font and color, and formatting for numbers and special types of data
- Behavior attributes such as shortcut menus, event generation, drill-down to **Find** or **Add/Details** forms, and attributes for dynamically enabling or disabling a component
- Other attributes that are specific to a type of component

There are two paths to creating and editing property class extensions. While you are in **Design Mode**:

1. Click **System > Edit > Property Class Extension**.
2. Select a component bound to an IDO property that is derived from a class and under the Inheritance section, click the **ellipsis** next to the property class extension attribute.

Bind components to an IDO property

Binding a component

Just as a form may be bound to a collection, the result set returned by an IDO, a form component may be bound to a property on the IDO providing the collection. A form component's data binding is a property from the IDO that returns the form's collection. This relationship means the bound component can be used to display and edit the data binding property value of the currently selected IDO item.

For example, you may want to bind a component to the property of an IDO so that whatever data entered in that component will end up in the database. When you do this any data for that field will automatically show up in your component. By binding to the IDO, it does two things:

- Associates the data in the component to that property of the IDO
- Inherits any attributes from the property and property class extension

Grid and VerticalGrid components can be bound to a collection.

These types of components can be bound to properties on a collection:

- Edit
- Multiline edit
- List box
- Drop List
- Combo box
- Date combo
- Check box
- Radio button
- Tree
- Gauge

In the case of GridColumn and VerticalGridElements components, they must be bound to properties on the same collection bound by their containing Grid or VerticalGrid.

Binding components to properties

Components bound to key IDO properties have special behavior. These components are enabled for new IDO items in the collection and are disabled for existing items. This functionality prevents the user from attempting to modify a key field once an item is created. Even if a component is not bound to a key property, you can enable this behavior by setting the **Read-Only For Existing Records** property.

In some cases, a property's value should not be updated at the time a new IDO item is created in the collection. The system supports this behavior with the **Read-Only For New Records** property. When this property is set to **True**, the component is read-only for new items but not for existing items.



Buttons and other unbound components can also be disabled by setting the property.

Once the component is bound to the code property, it inherits attributes from both the property class and the property class extension.

Setting data bindings for a component

The following is a high-level overview of how to set data bindings for a component.

Begin by opening a form and entering **Design Mode**. Then, select a component. Next, click **Binding** on the **Component** property sheet.

You select the type of binding you want to use in the **Type** box by clicking the **ellipsis**.



Available data bindings vary with the type of component.

Then, you click **Edit** to specify the binding attributes and click **OK** to return to the property sheet.



Demo: Associate inherited attributes

This demonstration shows how to associate inherited properties.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 5.1: Associate inherited attributes

In this exercise, you will cover adding a component to a form which will inherit from the property class extension.

Before you begin:

- Ensure you have completed Exercise 4.1 because it provides data or configurations for this exercise..

Exercise steps

1. Open the **Customer Orders** form.
2. Remove filter-in-place, if applicable.
3. Enter **Design Mode**.
4. Click the **User Defined** tab.

5. Draw a **ComboBox** component. The **Component** sheet displays.
6. Type *CustNumCombo1* in the **Name** field of the **Component** sheet.
7. Select **object.CustNum** from the **Binding** drop-down list, in the **Data Source** section.
8. Click **Save**.
Note: You may see a warning pop up, click **OK** to continue.
9. Exit **Design Mode**.
10. Locate your newly added component.
11. Click the drop-down arrow in the component field to see the options associated with the bound data source.
12. Go back into **Design Mode**.
13. Click your new component. The **Component** sheet displays.
14. Expand the **Inheritance** section of the **Component** sheet.
15. Go to the **View Attributes Inherited** field to see the inherited attributes for the component.
16. Close the form.

Check your understanding

The levels of inheritance that can be used in the system are:



- a) Property Class
- b) Property
- c) Property Class Extension
- d) Component Class
- e) Component
- f) All the above.
- g) None of the above.



Unbound components can use all 5 levels of inheritance.

- a) True
- b) False



Inherited attributes cannot be changed once applied:

- a) True
- b) False



Lesson 6: Setting up list sources

Estimated time

1 hour

Learning objectives

In this lesson, you will:

- Explain the purpose of a list source.
- Explain the purpose of IDO Collection.
- Describe how to use inline lists.

Topics

- Create a list source
- Create an IDO list source
- Create an inline list

Create a list source

When you use components like list sources on your forms, they make it easy for users to select data from displayed lists. List sources can be list boxes, drop lists, combo boxes, enhanced combo boxes, and grid columns. The source of the data that is presented in this list is called the component's list source. The specifications for a list source determine what items show in the component's list.

A list source can be one of these types:

- IDO Collection
- IDO Method
- Inline
- Script

To create a list source, you must be in **Design Mode**. After you open a form and create a component that is a list source, you specify the data to be displayed by using the **Edit List Source Specification** dialog box. The **Edit List Source Specification** dialog box is used to set up and maintain the specifications for a list source associated with a component. The specifications for each list source will vary with the type of list source you select.

We will focus on IDO and inline list sources in this lesson. However, here is brief definition of the purpose of script list sources.

Script list sources are used when it is necessary to use a global script to populate the list. The script can populate the list using the standard system scripting Application Programming Interface (API). You define the global script that populates the list, any parameters to be passed to the script, and the columns that are displayed in the list.

Create an IDO list source

There are two types of IDO list sources that you can create. You can create an IDO Collection list source or an IDO Method list source. Let's look at the difference between these two types of list sources.

The IDO Collection list source is populated by querying a collection. Once a query is initiated, there are properties returned to the user. Those properties may be displayed as columns in the list. The first column for display is always the column that provides a value for the component. You can filter the data that is being returned from the collection.

The IDO Method list source is populated by a stored procedure method that returns a list of values. The properties constitute the method's returned list of values. You can set the parameters that are passed to the method. These may include literal values, properties, variables, or components. You also set how the **Columns to Display** IDO properties that display as columns in the list. The first column for display is always the column that provides a value for the component.

You set the attributes that determine the behavior of the IDO list sources in the **Edit Source Specification** dialog box.

The following is a high-level overview of how to create an IDO list source.

Begin by clicking the list type component. The **Component** sheet will display. Then, click **Data Source** to expand the section.

Next, click the **ellipsis** in the **List Source** field. The **Edit List Source Specification** dialog box will display.



The options are: **None**, **IDO Method**, **IDO Collection**, **Inline**, and **Script**. You must specify the attributes of your list source depending on which option you select in the **Type** field.

1. You must select either the **IDO Method** or **IDO Collection** type by clicking the drop-down arrow in the **Type** field. Once you choose the list source type, the first group of options will be activated.
2. You click the drop-down arrow in the **IDO** field under the **Method and Collection Only** section and select the IDO collection type from the list that displays. Then, you click the **Properties** button and the **Properties of Data Sources to Include** dialog box will display with a list of all the properties for the IDO you selected.

Next, you select the properties you want to include in the list source in the left box and click **Add** and order the properties by clicking **Up** or **Down**. Once you have ordered the properties, you click **OK** to return to the **Edit List Source Specification** dialog box.



If you elected to create an **IDO Collection**, you will see the **Filter etc.** field activated under the **Collections Only** section. If you wish to specify filter criteria, order by, group by, and distinct attributes for the collection query, click this button. The **Specify Collection Options** dialog box that displays is where you can set up filters for the data that is being returned to the component. The **Read Mode** options allow you to override the default value for the collection. It will apply only to this list source. You are also able to set keywords for **Filter**, **Distinct**, and **OrderBy** to sort the collection.

You now must select the columns to display in the **Select Columns to Display in List** dialog box which you access by clicking **Columns to Display** and then, you click **OK**.



To remove a property, select it, and click **Remove**.

Create an inline list

You can create an inline list source for a component. An inline list source has a hard-coded table of values as part of the list specification.

Inline is the list attribute description. The following attributes create the table of values and determine its behavior and appearance in the list:

- **Entries** – Specifies the table of values as rows, columns, and values.
- **Column for Value** – Specifies which column of the hard-coded table provides the value for the component.
- **Columns for Display** – Specifies which columns of the hard-coded table are displayed in the list. The column that provides the value need not be displayed. It is determined by its association with the displayed columns.

These attributes are in the **Edit List Source Specification** dialog box which you are familiar with from the previous sections. After you open a form in **Design Mode** and create a component that is a list source, specify the data to be displayed for the inline list.

The following is a high-level overview of how to create and edit an inline list at the component level.

Begin by clicking the list type component. The **Component** sheet will display. Then, click **Data Source** to expand the section. Next, click the ellipsis in the **List Source** field and the **Edit List Source Specification** dialog box will display.

You click the drop-down arrow in the **Type** field to select the type of list source. You should select **Inline**. Then, click **Entries** in the **Inline Only** section and the **Edit Inline List Entries** dialog box will display.



You can also type in a comma-delimited list of values in the **Entries** field.

You can click in the cell and type the value you want to have displayed for the list.



The values are interpreted as string names in the system.

Then, you click **New Col** or **Delete Col** to add or remove columns and click **New Row** or **Delete Row** button add or remove rows.



If a list value corresponds to a string name, the system uses the string value as the list value. If the list value does not correspond to a string name, the list value is treated as a literal.

You click **Column for Value** to indicate the column number value you wish to send back to the database and click **OK**.



You can also type in the column number in the **Column for Value** field.

Finally, you click **Columns to Display** to choose which columns to display in the component and click **OK** to return to the form screen.



Demo: Add combo boxes and input list source information

This demonstration shows how to add combo boxes and input list source information.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 6.1: Add combo boxes and input list source information

In this exercise, you will add two more combo boxes to the User Defined tab of the Customer Orders form. The new components will be filled with list source information that you will provide. One will be an IDO Collection list and the other will be an inline list.

Before you begin:

- Ensure you have completed Exercise 5.1 because it provides data or configurations for this exercise.

Exercise steps

17. Type the step.

Part 1: Adding components and associating list sources on a form

1. Open the **Customer Orders** form.
2. Remove the filter-in-place, if applicable.
3. Click **Design Mode**.
4. Click the **User Defined** tab.
5. Draw two **ComboBox** components.

6. Click the first new combo box. The **Component** sheet displays.
7. Type *IDOlistComb* in the **Name** field of the **Component** sheet.
8. Click the second new combo box. The **Component** sheet displays
9. Type *InLinelistCombo* in the **Name** field of the **Component** sheet.
10. Click the **IDOlistComb** component.
11. Click the **ellipsis** in the **List Source** field in the **Data Source** section. The **Edit List Source Specification** dialog box displays.

Part 2: Edit List Source Specification dialog box input

1. Select **IDO Collection** from the **Type** drop-down list.
2. Select **SLCustomers** from the **IDO** drop-down list.
3. Click **Properties**. The **Properties of SLCutomers to Include** dialog box displays.
4. Select **CustNum** from the list of properties and then click **Add**.
5. Select **Name** and then click **Add**.
6. Click **OK** to return to the **Edit List Source Specification** dialog box.
7. Click the **Filter etc.** button.
8. Type *CustSeq=0* in the **Filter etc.** field.
9. Click **OK** twice to return to the form.
10. Click **Save**.
11. Exit **Design Mode**.
12. Locate the **IDOlistComb** component.
13. Click the drop-down arrow in the **component** field to see the options associated with the IDO collection list source.
14. Go into **Design Mode**.
15. Click the **InLinelistCombo** component.
16. Click the **ellipsis** in the **List Source** field in the **Data Source** section. The **Edit List Source Specification** dialog box displays.

Part 3: Edit List Source Specification dialog box input.....continued

1. Select **Inline** from the **Type** drop-down list.
2. Click the **Entries** button.
3. Click the **New Col** button to add a new column.
4. Add the following entries for the inline list:

Rows	Column 1	Column 2
Row 1	1st Quarter	01/01/2014

Row 2	2nd Quarter	04/01/2014
Row 3	3rd Quarter	07/01/2014
Row 4	4th Quarter	10/01/2014

5. Click **OK**.
6. Set the **Column for value** = 2.
7. Set the **Columns to Display value** = 1.
8. Click **OK**.
9. Click **Save**.
10. Exit **Design Mode** and click the drop-down arrow in the **component** field to see the inline list source.
11. Close the form.

Check your understanding



A List Source makes it easier for users to select data from displayed lists.:

- a) True
- b) False



A list source can be one of these types.

- a) IDO Collection
- b) IDO Method
- c) Inline
- d) Script
- e) None of the above



To create a List you must be in **Design Mode**.

- a) True
- b) False



An inline list source has a hard-coded table of values as part of the list specification.

- a) True
- b) False



Lesson 7: Adding validators to components

Estimated time

.5 hours

Learning objectives

In this lesson, you will:

- Explain the purpose of validators.
- Identify the types of validators.
- Describe how to create a validator.

Topics

- Explore the use of validators in the system
- Create, copy, edit, or delete validators

Explore the use of validators in the system

The system provides you with an extensive facility for validating values in components. Validators are global objects that can be reused across multiple components and forms. You can associate one or more validators with a component.

Validators can be executed when a user selects from a list or exits the component, or they can be deferred until the user attempts to navigate to a different IDO item in the collection or to save the form. They support a set of behaviors that allow you to validate a label on a component. For example, you can set up a validator that:

- Is related to the component list source in such a way that any value matching an item in the list is automatically considered valid.
- Validates a value and return output values to the form for functions like displaying associated descriptions or setting down-stream component defaults.

For you to use validators for a component, you must be in **Design Mode**. Once you open a form and select a component you will find **Validation** under the **Data Source** section on the **Component** sheet. When you expand **Validation**, you will see the associated properties.

To use a validator on a component, you must indicate which validator you want to associate with the component in the **Validators** field. You can indicate the validator properties in the **Edit Validators** dialog box which is accessed when you click the **ellipsis** in the field.

You can create, add, edit, or delete a validator in the Edit Validators dialog box. In addition, you can set parameters to substitute for the keywords in the error message for the validator.

Setting validator properties

Once you have identified the validator for the component, you must set the action that occurs when the user interacts with the component. You use the **Validate Immediately** property to set the timing for the execution of the validator. If you set the field to **True**, the validation occurs immediately when the user tabs off the field. If it is set to **False**, the validation occurs when the user attempts to save the record or navigates to another record.

You also must indicate the error message to display to the user on the **Message on Status Bar**. You can specify a particular string for the error message by selecting **True**. If you leave it set to the default **False**, it will display a message box that prompts the user to correct or ignore the error. You must specify an error message for all validators.

Using validator messages

As mentioned before, you can designate substitution keywords for validator messages. These messages strings may contain the following substitution keywords:

- **%V** is the value of the current component.
- **%L** is the label of the current component.
- **%B** is an error message passed from a validation stored procedure in a message type parameter.

Understanding the most common types of validators

There are different types of validators that you can use. However, the most commonly used ones are:

- **Alphanumeric validators:** Alphanumeric validators test whether the value entered is an alphabetic character or a numeral. Any text with other types of characters is rejected.
 - For example, you may use the following attributes to qualify a validation to specify that the value in the component should be greater than a specific number: `V% > 31`.
- **DateTime validators:** DateTime validators test whether the component's value has a valid date and/or time format per local standards.
 - For example, you can use `V% <= CURDATE( )` as a validator.
- **Inline list validators:** If the component has an inline list source, you can create an inline list validator that ensures that the value of the component matches one of the values of the inline list. Inline list validators can also output any column of the matched list item to a variable, property, or component.
 - For example, you can use `%V is not a valid %L` to check again inline list values.
- **In Collection validators:** In Collection validators can be used to determine whether the component's value exists as the value of a property in the IDO that returns the specified collection. Both the IDO name and the property to be matched must be specified as part of the validator definition.
 - For example, you can use `%V is not a valid %L` to check against a collection list of properties.

Considerations when using validators

You should consider the following, when you use validators:

- A validator's specification may reference a component, a variable, or a property value.
- If the property value changes after validating the component, then the component is marked as not validated. The validator would execute again if the user tabbed through the component or saved the form.
- For components whose values should be referenced against a table of values, use the collection-based list sources and validators.
- Use the **Validate Immediately** attribute to eliminate delays from performing all validation processing at save time (a behavior that is friendly to run-time users).
- Use component classes to package your validators, list sources and other component processing for reuse.
- Previously validated data may become invalidated because some value on which the validation is dependent changes.

Create, copy, edit, or delete validators

As you know, to create, copy, edit, or delete validators, you must be in **Design Mode**. You also know that you can create a validator by setting the properties within the Component Sheet.

The following is a high-level overview of how you can also use the system menu to create, copy, edit, or delete validators before you open a form.

Begin by clicking **Design Mode**. Then, select **System > Edit > Validators**. The **Validators** dialog box will display with a list of all validators that exist in the forms database.

You now have the option of performing one of these actions:

1. Add a new validator.
 - Click **New** to add a validator. The **Validator Properties** dialog box will display. Then, you enter the specifications for the validator. You click **OK** to return to the **Validators** dialog box.
2. Select a validator name.
 - Click **Copy** to copy the validator. Then, you enter a unique name for the new validator in the **Validator Properties** dialog box and adjust any other specifications. Click **OK** to return to the **Validators** dialog box.
3. Select a validator name.
 - Click **Edit** to edit the validator. The **Validator Properties** dialog box displays. You then update the specifications for the validator. Then, click **OK** to return to the **Validators** dialog box.

Once you have performed the action, click **Done** to close the **Validators** dialog box.



You can also delete a validator by selecting it from the list and clicking **Delete**.

Assign a validator to a component

The following is a high-level overview of how to assign a validator to a component.

Begin by opening a form in **Design Mode**. Then, click the component and the Component sheet will display.

You scroll to Data Source and click **Validation** to expand the property options. Next, you click the **ellipsis** in the **Validators** field and the **Validators** dialog box will display.

You click **Add** to display the list of validators, select a validator from the list, and click **OK**.



You can also edit the validator if necessary by clicking **Edit**.

The InInLineListGeneric validator has previously been created and you are just adding it to your component. Next, we will look at some of the attributes that were previously associated with this validator.



Demo: Add an inline list validator to the combo box form component

This demonstration shows how to add an inline list validator to the combo box form component.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 7.1: Add an inline list validator to the combo box form component

In this exercise, you will add an inline list validator to the combo box form component, which you created in an earlier exercise.

Before you begin:

- Ensure you have completed Exercise 6.1 because it provides data or configurations for this exercise.

Exercise steps

1. Open the **Customer Orders** form.
2. Remove filter-in-place, if applicable.
3. Click add an inline list validator to the combo box form component, which you created in an earlier exercise.
4. Select the **User Defined** tab.
5. Select the **ComboBox** with the associated list source of quarters.
6. Add **InInLineListGeneric** as the validator in the **Validators** section of the **Component** sheet.
7. Select **True** in the **Validate Immediately** field.
8. Save the form and test your component's validation.

Check your understanding



Explain the purpose of validators.





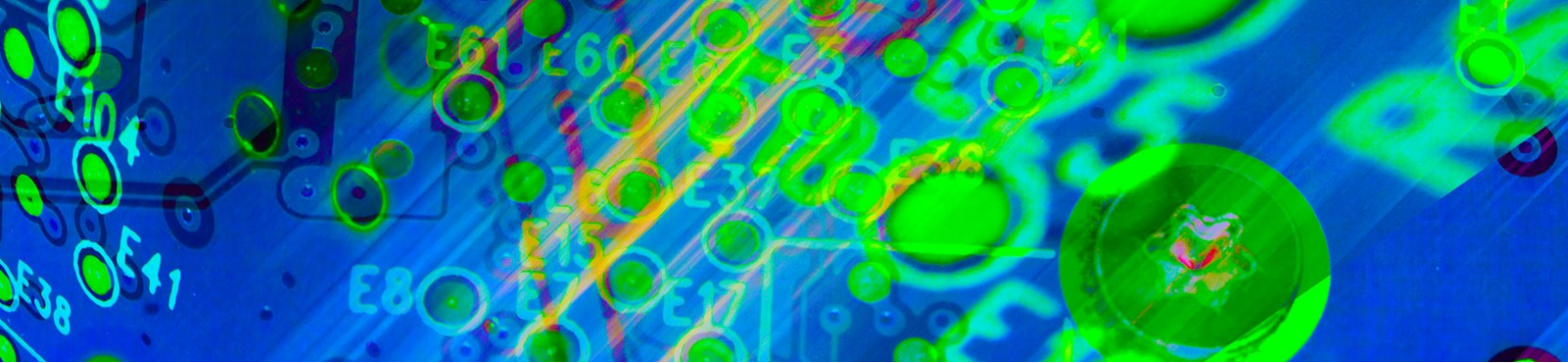
Identify the most commonly used types of validators, select all that apply.





To create a Validator you must be in **Design Mode**.

- a) True
- b) False



Lesson 8: Creating and editing strings

Estimated time

.75 hours

Learning objectives

In this lesson, you will:

- Explain how strings are used.
- Describe how the static component is used in strings.
- Describe how to create a string.
- Describe how to view and edit a string.

Topics

- Explore the use of strings in the system
- Use the static component
- Create a translatable string
- Copy, edit, or delete strings

Explore the use of strings in the system

A string is a type of global object that contains a translated literal. When you use string objects rather than literal text as form or component captions you ensure consistency and reduce the need for customization and translation.

For example, changing the value of the string named **sPartNo** from “Part No.” to “Part Number” affects all the forms and components utilizing this string. If literal text is used for each caption, each form and each component must be updated individually.

Strings are stored in a strings table in the forms database. The forms database can have multiple versions of the strings table. One version of the strings table is current at run time.

You can create or maintain string objects by using the **Edit/Select String** dialog box in the system. The following actions can be performed:

- Create a new string object.
- Copy the selected string object.
- Edit the value of an existing selected string object.
- Delete the selected string object.

Strings allow for the reuse of captions throughout the entire the system application, including reports.

Examples of strings

Strings are used extensively throughout the system. You will find them used primarily in four areas:

Use	Prefixed with
Static label captions	s (sItem)
Form captions	f (fCustomerOrders)
Messages	m (mIsNotAValid)
Text	t (tChangePOStatusUtility)

Use the static component

A static component may have a string name as its caption attribute. If it does, the value of the string will be displayed. Statics are often associated with other types of components like Edits or ComboBoxes and can inherit their captions.

To associate a static with another component and inherit its caption, the other component's caption is C(<staticcomponentname>) and the static has no caption attribute.

Create a translatable string

You can create a translatable string in **Design Mode** with no forms open, by selecting **String** from the **Edit** menu, and then clicking the **New** button in the Strings dialog box.

The following is a high-level overview of how you can also add a string value for a component.

Begin by going to **System > Edit > String**. Then, click **New** and type a *<new name>* for the string in the **Name** field.

You can add a string value in the **String Value** field, click **OK**, and click **Done**.

The string is now available to be used by any static label component in the entire system.

Copy, edit, or delete strings

The following are high-level overviews of how to copy, edit, or delete strings.

Begin by entering **Design Mode**. Then, click **String** on the **System > Edit** menu.

You can choose and perform one of these sub-procedures:

- To copy a string, you select a string name and click **Copy**. Then, specify a new name in the **Name** field, edit the **String Value**, click **OK**, and click **Done**.
- To edit a string, you select a string name and click **Edit**. Then, edit the **String Value**, click **OK**, and **Done**.
- To delete a string, you first select a string name. Then, you click **Delete**, click **OK**, and click **Done**.



You can create many new strings from this dialog box.



Demo: Create static form components and strings

This demonstration shows how to create static form components and strings.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 8.1: Create static form components and strings

In this exercise, you will create static form components and strings.

Before you begin:

- Ensure you have completed Exercise 7.1 because it provides data or configurations for this exercise.

Exercise steps

1. Open the **Customer Orders** form.

2. Remove the filter-in-place, if applicable.
3. Enter **Design Mode**.
4. Click the **User Defined** tab.
5. Add two **Static** components that describe or caption the two **ComboBox** components from the earlier exercise.
6. Type *<a name>* for the **Static** labels.
7. Link them to each combo box component.
8. Select an existing caption from the **Caption** field or create our own.
9. Close the form and reopen it to test the linkage between the **Static** component and the **ComboBox** components by entering incorrect data into the field of one of the **ComboBox** components.

Check your understanding



Explain the benefits of strings in the Infor Mongoose framework.





Identify how strings are used.





Describe how to create a string.





Lesson 9: Adding user-extended tables (UETs) fields and user-defined type lists

Estimated time

.75 hours

Learning objectives

In this lesson, you will:

- Identify the purpose of user-extended tables (UETs).
- Explain how to enter and use the user-defined type list feature.

Topics

- Add user-extended table fields
- Create and use a new user-defined type list

Add user-extended table fields

The user extended tables (UET) feature gives the system administrator the ability to extend existing application database tables and add custom user fields to forms in the application. Use this feature to keep track of information that is not currently in the application database schema.



If you are using replication, you must click the **Regenerate Replication Triggers** button on the **Replication Management** form after UETs are changed, added, or deleted.

Tables are a systematic arrangement of data in records and fields for ready reference. The application ships with tables containing predetermined fields. The UET feature allows users to add their own fields to these tables. Once a table is extended, you can drop user fields into any application form that uses this table.



If you bind a new component to a UET on a form that uses a custom load method stored procedure, an error message displays when you refresh the form. However, the error does not prevent you from continuing.

Adding user-extended table fields

Use the **UET User Fields** form to create new user fields, modify existing user fields, or delete existing user fields for UETs. Then use the **UET Classes** form to create new user classes, modify existing user classes, or delete existing user classes for UETs.

1. Use the **UET Impact Schema** form to apply new and changed definitions of User Extended Tables to the schemas in all affected databases. The **UET Impact Schema** form creates physical storage for User Extended Table extensions when you make changes. These changes include creating new extensions to a database table, or updating a property of a user index or user field.



The Impact Schema process regenerates triggers for changed tables. To prevent blocking, run the process when no other users are logged on the database.

2. After the schema is impacted, restart the application session to refresh the session cache using the new schema information.
3. On the **UET Impact Schema** form, select **Commit Form Changes** to save the changes you have made to the form information using the UET forms.
4. Select **Impact Schema** to change the database schema to contain the columns and indexes corresponding to the user fields you have defined in the UET forms. Corresponding views over multi-site tables are also updated. Then, click **Process**.

Create and use a new user-defined type list

A user-defined type specifies values for a user-defined property. For example, if a user creates a user-defined property for color, the user could also create a user-defined type that specifies the values red, blue, and green. The user-defined type supplies a valid list of values for the property.

User-defined type refers to a named group of data values that can be used as a list source for a component.

- For example, a user-defined type named "Size" could invoke a list of values consisting of "Small," "Medium," and "Large."

Users can create user-defined types and add values to them in the **User Defined Types** form. Once a user-defined type is specified as the source for a component that supports lists, users can add values to the type directly through the right-click Add function.

- In the example, above, a user might add the value "X-Large."

User-defined types are useful with user-defined fields and user-extended tables. Users can add list-type components to forms without modifying the schema or IDOs or writing special form code. **Value** and **Description** pairs display in a two-column list.



The **Add User-Defined Type** dialog box is available only from the **New Data Maintenance Wizard**.

You can create reusable list pair values that can be used for selection data in a combo box. **User Defined Types** allow for any user list that is unique to a customer or situation.

The benefits of the **User-Defined Types** are convenience, flexibility, and the reusability. For example, they can be special values for a particular part, a unique list of codes or status, or a special list of specs or dates.

To enter and use the user-defined type list, use the **Component Class UserDefinedType**.



Demo: Add UET field and data

This demonstration shows how to add a UET field to a database table, and then associate a user defined attribute table to this field for validation and entry.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 9.1: Add UET field and data

In this exercise, you will add a UET field to a database table, and then associate a user defined attribute table to this field for validation and entry.

Before you begin:

- Ensure you have completed Exercise 8.1 because it provides data or configurations for this exercise.

Exercise steps

Part 1: Add a UET Class

1. Open the **UET Classes** form.
2. Start a new record and name it **ItemSpecs**.
3. Type a general description in the **Label** field.
4. Save the record.

Part 2: Add a UET Field to a table

1. Open the **UET User Fields** form.
2. Click a new record.
3. Add a record for UFCOLOR, UFLength, and UFWIDTH.
4. Use a Data Type of 'nvarchar' and a Length of 20 for UFCOLOR.
5. Use a Data Type of 'int' for UFLength.
6. Use a Data Type of 'int' for UFWIDTH.

Note: You may use a Data Type of decimal for UFLength and UFWIDTH if you so desire.

Part 3: Associate a UET Field to a Class

Use the **UET Class/Field Relationships** form to associate user fields with a user class.

1. Open the **UET Class/Field Relationships** form.
2. Click the **Refresh** button and begin adding new records.
3. Select **ItemSpecs** from the **Class Name** fields. Perform this task three times and each time select one of the three new **User Fields** that you just created.

4. Save the records.

Part 4: Associate a UET Table to a Class

1. Go to the **UET Table/Class Relationships** form.
2. Click the **New Record** button. Once this button is selected, you can begin associating your new class with a table in the system.
3. Select the **co_mst** table and the **ItemSpec** class from the drop-down options.
4. Save the record.

Part 5: Impact the Schema of a Table

1. Open the **UET Impact Schema** form and click **Process** to impact the schema.
2. Select **System > View > User Preferences > Runtime Behaviors**.
3. Confirm the **Unload IDO Metadata With Forms** check box is selected.
4. Select **System > Form > Definition > Unload all Global Form Objects**. This is critical to ensuring that the new properties will be accessible in the **Design Mode**.
5. Close out of and restart the system.

Part 6: Add and Bind the newly created UET Fields to a form

1. Open the **User Defined Types** form and enter at least one new record. Each record should be the descriptive values for **UFColor**, **UFLength**, and **UFWidth**.
2. Name the new **User Defined Types Colors, Lengths, and Widths**.
3. Draw a **ComboBox** on the **Customer Order** form.
4. Type *UFColorCombo* in the **Name** field.
5. Click **Binding** on the **Binding** tab.
6. Select **Properties** from the drop-down list.
7. Select the **Customer Orders** collection.
8. Scroll down to **coUfColor**, **coUFLength**, **coUFWidth**. The **User Extended** table fields are prefixed with “co”
9. Bind this component to **coUfcolor**.
10. Click **OK**.
11. Associate the **ComboBox** component data to the **User Defined Data Type Color** that you created in step 20.
12. Select **UserDefinedType** from the drop-down box on the **Component Class** combo box located under the **Inheritance** node.
13. Type *Colors* in the **Parameter** field.
14. Click **OK** to confirm your action.
15. Save and reload your form. Notice that now you can save this new data into the UET field by record.



Demo: Work with a UET

This demonstration shows how to work with a UET.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.





Exercise 9.2: Work with a UET

In this exercise, you will take advantage of either the User Defined Type or the list source combo box created in earlier exercise. You will need to have the Customer Orders form open.

Before you begin:

- Ensure you have completed Exercise 9.1 because it provides data or configurations for this exercise.

Exercise steps

1. Open the **Customer Orders** form.
2. Go into **Design Mode**.
3. Click the **User Defined** tab.
4. Select one of the combo boxes from the previous exercises.

Note: You have the option of choosing the combo box holding your IDO Collection list source data, the combo box holding your inline list source data, or the combo box holding your User Defined Type List from the previous exercise.

5. Select a **User Defined Field** to save this data.

Note: The option you select will depend on the combo box you chose since you must know the data-type and length of the data that is associated with that component.

6. Click **Save**.
7. Close the form.
8. Re-open the form.
9. Return to the combo box for that component.
10. Select a piece of data from the combo box drop-down list for any of the available records.

11. Save the form.
12. Close the form.
13. Reopen the form and refresh the form and data, or navigate off the record then back again to see the data stored in the collection and on the database.

Check your understanding



The user extended tables (UET) feature gives the system administrator the ability to extend existing application database tables and add custom user fields to forms in the application.

- a) True
- b) False



User-defined type refers to a named group of data values that can be used as a list source for a component.

- c) True
- d) False



Explain how to enter a user-defined type list.





Explain how to use the user-defined type list feature.





Lesson 10: Setting up event handlers

Estimated time

.75 hours

Learning objectives

In this lesson, you will:

- Explain the purpose of event handlers.
- Identify the types of event handlers that are used in forms.
- Describe how to add an event handler to a form.

Topics

- Explore the use of events and event handlers in the system
- Create, copy, edit, or delete event handlers

Explore the use of events and event handlers in the system

You can define events to be generated when the user interacts with a component. Most components support a primary event, a data change event, a gain focus event, and a lose focus event.

For instance, you might want to generate a custom event when the user clicks a push button. To do this, specify the custom event name as the push button's primary event. Examples of push button action would be clicking a button that:

- Prints a report.
- Opens another form.
- Launches another application.
- Runs a special program or script.

The table below describes the conditions under which each type of event is generated.

Event/Condition	Description of outcome
Edit/double-click	The data in the edit component has been modified, and the user tabs or clicks out of the component. The user tabs or clicks into the component. The user tabs or clicks out of the component.
Multiline edit/double-click	The data in the edit component has been modified, and the user tabs or clicks out of the component. The user tabs or clicks into the component. The user tabs or clicks out of the component.
List box/double-click	The user selects an item from the list that is different from the previously selected item. The user tabs or clicks into the component. The user tabs or clicks out of the component
Drop list/double-click	The user selects an item from the list that is different from the previously selected item. The user tabs or clicks into the component. The user tabs or clicks out of the component.
Combo box/double-click	The user modifies the data in the edit box and tabs or clicks away. Or the user selects an item from the list that is different from the previous value of the component. The user tabs or clicks into the component. The user tabs or clicks out of the component.
Date combo/double-click	The user modifies the data in the edit box and tabs or clicks away. Or the user selects a date from the drop-down calendar that is different from the previous value of the component. The user tabs or clicks into the component. The user tabs or clicks out of the component.
Push button/click	The user tabs or clicks into the component. The user tabs or clicks out of the component.
Check box/click	The user tabs or clicks into the component. The user tabs or clicks out of the component.
Radio button/click	The user tabs or clicks into the component. The user tabs or clicks out of the component.

Event/Condition	Description of outcome
Notebook/tab	The user tabs or clicks into the component. The user tabs or clicks out of the component.
Grid/double-click	The user tabs or clicks into the component. The user tabs or clicks out of the component.

Understanding event handlers

There are two types of events and event handlers in the system: form events and application events. They are similar, but the key distinguishing factor, though, is whether they are form-only in scope or application-wide. This lesson focuses on form events.

Event handlers define a unit of work to be performed in response to an event. Event handlers are defined for specific forms. They are not shared across forms. For example, the types of event handlers that can be used on a form are:

- Goto URL
- Run Form as Linked Child

Event handler sequence

Event handlers are identified by the name of the event that they are handling and by a sequence number. You can define multiple event handlers for any event. These event handlers execute in sequence number order.

By default, if any event handler reports a failure, the remaining event handlers in the sequence are skipped. You can override this default by setting the **Ignore Failure** parameter. If you set this attribute, the event handler will always be treated as successful.

Built-in responses

When you define additional event handlers for standard events that have built-in responses, the built-in responses always execute last. Consequently, you can cancel the built-in responses by reporting failure from a custom event handler for the same event.

There are many different response types to use as event handlers. The main categories are:

- Collection processing
- Execute an executable
- Form navigation
- Generate event
- Goto form
- Inline script
- Method call
- Run background task
- Run script
- Set values

Event handler messages

For any event handler, you can define messages to be displayed when the event handler fails or succeeds. You use the **Error Message** and **Success Message** parameters to define each message.

The message definition may contain a string name and may also reference any of the standard substitution keywords.

- For example, you can use the %B substitution keyword to reference a message type parameter that is defined for an event handler of the **Method Call** response type.

By default, messages generated from event handlers are presented in a pop-up dialog box. Alternatively, you can specify that they be presented in the application status bar.

- To do so, use the **Message on Status Line** parameter.

If you are using multiple collections on a form, you may write event handlers that are specific to each collection.

- To do so, use the event handler's **Only When Current Collection Is** parameter.

When you specify a specific collection for an event handler, form-based events will be handled by this event handler only when the specified collection is active. Collection-based events will be handled by this event handler only when the event is generated by an operation on the specified collection.

The **Event Handlers** dialog box is used to create, copy, edit, delete, enable or disable event handlers for the current form. You can also change the order in which event handlers execute and find strings in event handler specifications.

The content of the event handlers list depends on how you open the dialog box:

- If you open the dialog box from the **Events** sheet for the component, with no event name specified, the list displays all event handlers for the form.
- Similarly, if you open the dialog box from the WinStudio menu bar by selecting **Edit > Event Handler**, the list displays all event handlers for the current form.
- If you open the dialog box from the **Events** sheet with an event specified, this dialog box lists only the event handlers that are associated with the specified event. In this case, the title bar of the dialog box identifies the event whose event handlers are listed.

Create, copy, edit, or delete event handlers

So now you know that event handlers are objects specific to a form that supplement, alter, or cancel the default event-handling behavior of system. You create and name event handlers, which execute when an event matching the name of the event handler is generated. The event can be generated as the user interacts with a form, or it can be generated programmatically by the developer.

Create, copy, or delete event handlers for a form

The following is a high-level overview of how to create, copy, or delete event handlers for a form.

Begin by opening the form you want and entering **Design Mode**. Then, click **Event Handlers** on the **Edit** menu. The dialog box will list any event handlers created for the current form.

Next, you perform one of these actions:

- To add an event handler, you click **New**. Then, specify the event for which you are creating an event handler, the sequence of the event handler, and the response type of the event handler. Next, you click **OK**.
- To copy an event handler, you select the one you want and click **Copy**. Then, you enter a unique name for the new event handler and adjust any other specifications. Next, click **OK**.
- To delete an event handler, you select the event handler, and click **Delete**. Then, confirm the deletion.

Once you have completed the action, click **Done**.

Edit an event handler

Event handlers on the base form are grayed out, indicating that they are not available for editing on this form.

The following is a high-level overview of how to edit the event handler for the extended form while in **Design Mode**.

Begin by selecting it and clicking **Edit**. Then, click **Event Handlers** and the **Event Handler Properties** dialog box will display.

Next, you set the **From Base Form** property to **False** and edit the event handler.

This causes the event handler to be associated with the extended form. Be aware that, if you have a base form event with multiple handlers, and you edit one handler this way, all the other handlers defined for that event also become associated with the extended form.

To specify a single event for which you want to view the defined handlers, use the drop-down list. To see the description of each event handler, review the table below.

Column	Description
Event	Indicates the internal name of the event for which the handler is defined
Seq	Indicates the order in which the event handler executes
Type	Indicates what type of event handler is defined
Disabled	When selected, indicates that the event handler is not to be executed

Column	Description
Response	Indicates how the system is to respond when the event handler executes
Dialog box options	This typically consists of a set of parameters that define what the handler is to do This dialog box includes these action buttons: • Event filter –To view all event handlers, select [ALL]. • OK – To finish working in the dialog box and save your changes, click this button. • New – To create a new event handler, click this button. • Copy – To copy an event handler, select the handler to copy and click this button. • Edit – To edit an existing event handler, select the handler and click this button. • Delete – To delete an event handler, select the handler and click this button. • Cancel – To cancel any changes made and close the dialog box, click this button. • Handlers – To filter the list of event handlers for the event generated by a handler of the type, click this button. • Find – To search the list of event handlers for a particular handler, click this button. • Up and Down arrows – To change the sequence order of event handlers, use these buttons. • Help – To view the help topic for this dialog box, click this button.



Demo: Create a new button form component and add an event handler

This demonstration shows how to create a new button form component and add an event handler.

Before you begin:

- Ensure you are logged in to [application] and are at the [location in app] for [user account]. If not, refer to Exercise 1.1.
- [Other prerequisite or note].
- To prepare the system for future exercises, this demo must be completed.





Exercise 10.1: Create a new button form component and add an event handler

In this exercise, you will create a new button form component and add an event handler. This exercise will require you to move past the material presented in the example. You will be required to add an event that will launch a linked child form.

Before you begin:

- Ensure you have completed Exercise 9.2 because it provides data or configurations for this exercise.

Exercise steps

Part 1: Create a new button on a form

1. Open the **Price Matrix** form.
2. Remove the filter-in-place, if applicable.
3. Click **Design Mode**.
4. Select *Site Default* from the **Scope** drop down list.
5. Click **OK**.
6. Click **OK** to the message asking you to Unload Global Objects.
7. Click **Ctrl+U**.
8. Add a **Button** component.
9. Type `<a name>` for the button.
10. Type *sPriceFormulaLinked* for a caption.

Note: You may need to resize the button for the caption to fit and create the string.

Part 2: Add an event handler to the button

1. Click **Events** on the **Component** property sheet.
Note: This button looks like a lightning bolt.
2. Type *LaunchPriceFormulaLinked* on the **Primary** button as the name of the new event.
3. Click the ellipsis on the **Primary** field.
4. Click **New** on the **Event Handler** form.

5. Select **Run Form as Linked Child** from the **Response Type** drop-down list.
6. Click the **ellipsis** on the **Parameters** field.
7. Click **Type Specific Parameters**.
8. Select the **Price Formula** form as the form to launch.
9. Tab off the **Form** dialog box to enable the **Link By** button.
10. Select the **Priceformula** property from the **Parent Property** field.
11. Select the **Priceformula** property from the **Child Property** field.
12. Click the **Add** button.
13. Click **OK** until all open forms are closed and you are left with the **Price Matrix** form in design mode.
14. Click **Save**.
15. Exit **Design-Mode**.
16. Click **CTRL+U**.
17. Open and test the **Price Matrix** form.



Make sure to add your new string caption to the strings table and test your linked form. You can do this by placing one form above the other and scroll through the records on the **Price Matrix** form.

Check your understanding



Explain the purpose of event handlers, select all that apply.





Identify the types of event handlers that are used in forms.





Describe how to add an event handler to a form.





Lesson 11: Form Sync

Estimated time

.75 hours

Learning objectives

In this lesson, you will:

- Explain the purpose for Form Sync.
- Identify the Key areas of Form Sync.
- Describe how to carry out Form Sync.

Topics

- Explore the use of the Form Sync Form.
- Move customizations from one configuration to another.

Why you need FormSync

As a Systems Administrator, you have been assigned the task of making some significant changes to a form. As best practice dictates, you would not start to make changes to the form in question in your production environment. Hence, you would log into your test environment and commence the process of making the relevant changes and carry out the testing of those changes.

The next step in the process would be to move all the changes you made into your production environment without having to duplicate your efforts. This is where FormSync becomes an invaluable tool in that with the click of a single button, you are able to duplicate all your changes from a source system to a target system.

Overview

In Design Mode, WinStudio serves as a development tool for customizing forms in Mongoose-based applications. FormSync is an aid to preserving your customizations when you upgrade to a newer version of Mongoose or your Mongoose-based application; when you apply a service pack or a single issue fix; or when you distribute and apply your own forms, form modifications and other global objects in Mongoose. The process of handling these changes during an update or upgrade is referred to as *synchronization*.

Note: Mongoose also incorporates much of the functionality described in this guide within the **Form Sync** form. But the reader should be aware that not everything described in this guide applies equally to that form. For more information about the differences, see the online help for that form.

FormSync also provides a set of development and administrative utilities with which you can:

- Gather information about customizations.
- Copy customizations from one database to another.
- Delete objects.
- Compare objects.
- Generate SQL scripts to apply customizations to a remote database.
- Generate "pseudo code" for forms and objects to use later in evaluating and updating customizations.

The Nature of Customizations

FormSync looks for differences between a source version and a target version. Any type of change that has been made to an attribute value registers as a customization in synchronization, whether or not the change affects functionality.

For example, if a customer developer adds a comment to a script, FormSync considers the script to be customized, even though the functionality of the script has not changed.

A set of specifications for a complex object, such as a component class or a script, is treated as a unit in synchronization. Mongoose does not attempt to selectively synchronize individual specifications for the object.

The relationship between Global Objects and Forms during customization

Global objects are listed as attributes of forms and components in the Merge Options dialog box. The values of these attributes are named references to global objects. A customization of a form component with regards to global objects consists only of selection of a different named object in the component specifications.

For example, if a component uses the right-click menu named StdDefault and a customer developer changes the value for a component to StdDetailsAddFind, FormSync processes the change as a customization of the component, not of the form that uses it. Customizations of global objects themselves are processed separately, as specified in the Synchronization Options dialog box. If you change the attributes of the menu StdDefault, the menu is processed as a customized version of the global object, but form components that use the customized menu are not processed as customized.

The Basic Synchronization Process

For any event handler, you can define messages to be displayed when the event handler fails or During synchronization, FormSync compares new versions of forms and objects (in the Source database) with existing versions of those forms and objects (in the Target database). FormSync also compares the Source versions with any existing customizations in the Target database. Depending on your synchronization settings, FormSync then either keeps the customizations, merging them with new data from the Source database, or removes and replaces customizations with the new data.

Finally, FormSync replaces the existing vendor versions in the Target database with the new vendor versions from the Source database. For more information, see Chapter 2, "Schematic Overview of Synchronization."

Conflicts Between Customizations and New Versions

FormSync does not attempt to resolve conflicts between the size and position of customized components and revised settings for size and position in a new base version. Where you might have moved, resized, or added components on a form, those components can display overlapped with a base version of a component in a merged form.

Automatic Conversions of Scripts

One of the major changes for FormSync Version 8.0 (which has been carried forward to subsequent versions) was the conversion of VBA code to .NET code. During synchronization, FormSync evaluates both form and global scripts to determine whether they are VBA code, which was used in earlier versions of applications (such as SyteLine) built on Mongoose. If the code is determined to be VBA, then FormSync attempts to convert the code automatically to VB.NET. After converting the code, FormSync prompts you to keep, remove, or edit the new version of the script.

If you choose to:

- **Keep** the converted code, FormSync commits the automatically converted code to the Target database.
- **Remove** the converted code, FormSync removes the customized script and replaces it with the script from the new (Source) version.
- **Edit** the converted code, FormSync opens the file comparison utility with the new Source version of the script in one pane and the customized code that FormSync converted to VB.NET in another pane. This allows you to compare versions, evaluate the new converted version, and make changes in the customized version if necessary, before FormSync commits the changes to the Target version.

Note: Whether you keep the converted code as is or edit and then save it, there are no guarantees that the resulting code is fully functional. Plan to test the script before placing it into a production environment.

FormSync does not automatically convert inline scripts. If you have inline scripts, FormSync simply removes them, replacing them with the new Source version. If you want to continue using the customized version, you must manually convert it after synchronization.

Scope of Synchronization

FormSync processes customizations made to the client tier only, that is, forms and global objects. Such customizations are made in WinStudio Design Mode. Common customizations include actions like:

- Resizing a form
- Adding a text-box to a form
- Changing the source for a drop-down list

FormSync does not process or examine changes made to the middle (IDO) tier or the application-database tier. Synchronization and the operations performed by FormSync utilities do not apply to objects listed in this table.

Objects Not Processed by FormSync

Tier	Object
Middle	IDO metadata Report definition files
Application database	Table columns Stored procedures Application Event System metadata Triggers Functions Report background-task definitions

Planning Customization and Synchronization

As a development project, customizing WinStudio forms requires planning, documentation, and application of development policies. FormSync cannot anticipate the consequences of all customizations or guarantee the functionality of both customizations and base-level functionality after an update is applied. FormSync is most effective in maintaining functionality when it is used in conjunction with a development plan, documentation supporting the plan, and testing.

FormSync Administration Utilities

The functionality of FormSync in the update process is different from its functionality as a development tool for administering formsdatabases.

In an update, synchronization handles vendor versions of forms in both the revised and the unrevised versions of a Mongoose-based application. FormSync compares any customized version of a form with the two vendor versions of it. FormSync then allows you to update the customized version with specifications from the new vendor version, while selectively keeping customizations. Alternatively, synchronization can remove customized versions entirely or keep them intact. In all cases, FormSync replaces old vendor versions with new vendor versions.

FormSync's utilities for copying, deleting, and scripting objects do not affect vendor versions of forms or global objects. These utilities access only customized versions. These utilities are useful primarily to those

who customize forms using a development forms database and subsequently copy the customizations to a production forms database. Utility operations do not alter the application version number or service-pack number in a forms database.

The utilities can also serve an informational role in updates. You can:

- Generate lists of customizations currently in a forms database.
- Compare customizations with vendor versions.
- Store customizations in script files before you synchronize databases.
- Generate "pseudo code" to use in evaluating and comparing objects.

For more information, see Chapter 6, "FormSync Utilities" and the online help for "Utilities."

Note: These utilities are available only with the standalone version of FormSync, and not with the Form Sync form.

Synchronizing Site Default or Group Versions

In addition to synchronizing customized versions with new vendor versions in the update process, you can synchronize customized versions in a production forms database with customized versions in a development forms database. This feature of FormSync allows customer developers to conveniently deploy customizations made at Site Default scope and Group scope.

For More Information

For more information about how FormSync works, including information about how to use FormSync effectively, see the online help for FormSync. We especially recommend that you read the following topics before using FormSync:

- "Understanding WinStudio Customizations"
- "About Synchronization" (and subtopics)
- "Before You Use FormSync" (and all subtopics)
- "Using FormSync" (and subtopics)



Exercise 11.1: Form Sync the Price Matrix form

In this exercise, you will FormSync the Price Matrix form that you customized in exercise 10.1. On completion of this exercise you will see your customizations carried forward to the target configuration.

Before you begin:

- Ensure you have completed Exercise 10.1 because it provides data or configurations for this exercise.

Exercise steps

Part 1: Create an Export XML file of the Price Matrix form

1. Make sure you are logged into the system using the **CSI Click Once Client**.
2. Log into the system with the credentials provided to you by your instructor.
3. Open the **FormSync** form.

4. Select *CSI_Demo_DALS* from the **Source** drop-down list.
5. Select *CSI_Demo_DALS* from the **Target** drop-down list..
6. Click the **OK** button.
7. Click **OK** to the message that will be displayed.
8. Click the **Utilities** button.
9. Select *Site* from the **Scope Type** drop down list.
10. Click the **Refresh List** button. You should see a list of all Site Default versions of forms displayed under the **Custom Objects** grid.
11. Click the **Deselect All** button.
12. Select the **Price Matrix** form by clicking the checkbox under the **Inc** column.
13. Select the **Export** Tab.
14. Make note of the filename path and the name of the .XML file. You may click the ellipsis  button and change the filename and path if you so desire.
15. Click the **Export** button.
16. Select **Yes** to the Form Sync message that displays.
17. Close the **Form Sync** form.
18. Log out of the system.

Part 2: Import the XML file of the Price Matrix form you created in Part 1.

1. Log into the **Target** configuration.
Note: For this exercise, we will log into the system using the **Train Click Once Client**.
2. Type *train01@gdeinfor2.com* in the **User ID** field.
3. Type *train01* in the **Password** field.
4. Select **Train01** from the **Configuration** drop-down list.
5. Click **OK**.
6. Open the **FormSync** form.
7. Select **Train01** from the **Source** drop-down list.
8. Select **Train01** from the **Target** drop-down list..
9. Click the **OK** button.
10. Click **OK** to the message that will be displayed.
11. Click the **Utilities** button.
12. Select the **Import** tab.
13. Click the **ellipsis** next to the **Filename** field and select the XML file you created in Part 1. You should see the Price Matrix form displayed in the **Import Objects** grid.
14. Click the **Import** button.
15. Click **OK** to the Form Sync message.

16. Click **OK** to the Import complete message.
17. Close the **Form Sync** form.
18. Press **Ctrl+U**.
19. Open the **Price Matrix** and make sure your customizations have been applied to the system.

Check your understanding



FormSync is an aid to preserving your customizations when you upgrade, apply a service pack or a single issue fix; or when you distribute and apply your own forms, form modifications and other global objects in Mongoose

- a) True
- b) False



FormSync resolves conflicts between the size and position of customized components and revised settings for size and position during synchronization.

- a) True
- b) False



FormSync processes customizations made to the client tier only, that is, forms and global objects. Such customizations are made in WinStudio Design Mode.

- a) True
- b) False



Course summary

Estimated time

.5 hours

Learning objectives

Now that you have completed this course, you should be able to:

- Identify the elements of the system data architecture.
- Identify how to assign different levels of form editing permission to specific users.
- Describe how to apply editing attributes to a form.
- Explain how to use inherited properties in forms.
- Identify the how to create list sources, IDO list sources, and inline lists.
- Describe the purpose of validators and how they are used in forms.
- Explain how to create and edit strings.
- Describe how to add user-extended tables (UETs) fields and user-defined type lists.
- Explain how to apply an event handler to a form.
- Explain how to use FormSync.

Topics

- Course review

Course review



What outstanding questions regarding upgrade implications do you have?





What questions regarding the content covered in the course do you have?





Appendices

The following are included in this section:

- Appendix A: User accounts
- Appendix B: Creating a new form

Appendix A: User accounts

Your instructor will assign you a student user ID from the table listed below to use for class exercises.

Note: If you are taking this course as self-directed learning, refer to the Training Desktop Login Instructions on the Lab On Demand page.

CloudSuite Industrial: v10 Creating Form Personalizations (v10.18) (01_0011018_IEN3552_SYT)			
Training Environment entry point (VM)	User	Password	
CSI 10 Desktop	EDUDesktop\Infor	Infor123	
GDECSIBE (Landing Server)	gdeinfor2\administrator	G!oba!08	
Application	User	Password	Employee Name
Instructor login (for course demos): • ADFS	osservice@gdeinfor2.com	G!oba!08	N/A
Instructor login (for course demos): • CSI • Factory Track	sa	G!oba!08	N/A
Student logins (for course exercises): • CSI	train01@gdeinfor2.com	train01	N/A

Appendix B: Creating a new form

Copy a form

Before anything is explained in too much detail, it is important to understand a common-sense guideline in creating new forms.

It is very rare that a form will need to be created entirely from scratch. In most cases, a similar, existing form can be copied and modified.

To copy a form, begin by opening a form in **Design mode**. Then, select **System > Form > Definition > Copy** from the main menu. The **Copy Form Definition** dialog box will display.

- Next, select the form that you wish to copy from the **Source Form** drop-down menu. Then, type *<a name>* in the **Target Form** field.
- Check the **Copy Template Also** check box to copy the template and click **OK**.

You can now search for, open, and modify the form to fit your needs.

Create a new form

The first thing that you should understand before creating a new form is the difference between bound and unbound forms.

A bound form is one that is used to retrieve and manipulate one or more collections. (A collection is the result set returned by an IDO.) The relationship between the form and an IDO is determined by specifying the collection to which the form is bound. In other words, a form is associated with the result set that is returned by a particular IDO.

An unbound form is one that is not tied to a particular IDO. It may be used to simply gather data to pass to a method or procedure, such as a report generating form.

To create a bound form, make sure you have Site Developer permission. Then, go into **Design Mode**.

- Next, select **System > Form > Definition > New**. The New Form Wizard screen will display.
- Select Create a New Form, click Next, and type the *<form name>* in the field.
- Add attributes in the **Add Attributes** section; this includes selecting a data source from the **Data Source** drop-down list.
- Then, click **Next**.
- Select properties for the form and then click **Next**.
- Finally, select the order of the attributes on the **Properties** screen and then click **Next > Finish**.